

FernUniversität in Hagen
Fakultät für Mathematik und Informatik
LG Kooperative Systeme

UDP Transport Options

Implementierung und Analyse von RFC 9868 in Rust

Masterarbeit
im Studiengang Master Informatik

vorgelegt von
Alan Bernstein

Matrikelnummer: 2068834

Erstprüfer: Prof. Dr. Christian Icking
Zweitprüferin: Dr. Lihong Ma

Hagen, 15.09.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
Abkürzungsverzeichnis	vi
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum	2
1.2.2 Unidirektionalität	2
1.2.3 Herausforderungen	2
1.3 Zielsetzung	3
1.4 Abgrenzung und Umfang	4
1.5 Aufbau der Arbeit	4
1.6 KI-Einsatz	5
2 Grundlagen	6
2.1 IPv4	6
2.2 Das User Datagram Protocol (UDP)	10
2.2.1 Protokollstruktur	10
2.2.2 Eigenschaften	13
2.3 Erweiterungsmuster im Vergleich	14
2.4 Die Surplus Area	15
2.5 Betriebssystem und Netzzpfad	21
3 Methodik und KI-gestützte Entwicklung	26
3.1 Forschungsdesign	26
3.2 RFC-Auswertung und schrittweise Umsetzung	28
3.3 Beteiligte und ihre Rollen	29
3.4 Bekannte Schwächen der Sprachmodelle	31
3.5 Testarchitektur und Prüfprozess	32
3.6 Externe Referenzfälle	35
3.7 Reproduzierbarkeit und Grenzen	36

4	RFC-Analyse und Anforderungsmodell	37
4.1	Anforderungen	37
4.1.1	Funktionale Anforderungen	37
4.1.2	Nicht-funktionale Anforderungen	37
4.1.3	Speichereffizienz und Zero-Copy	37
4.2	Existierende Implementierungen	37
4.2.1	Linux Kernel	37
4.2.2	Userspace-Implementierungen	37
4.3	Herausforderungen	37
4.3.1	Zugriff auf die Surplus Area	37
4.3.2	NAT-Traversal	37
4.3.3	Interoperabilität	37
4.3.4	Alternative Implementierungsansätze	37
4.4	Technologieauswahl	38
4.4.1	Programmiersprache Rust	38
5	Entwurf und Implementierung	45
5.1	Architektur	45
5.1.1	Architekturüberblick	45
5.1.2	Projektstruktur	45
5.1.3	Datenmodell der UDP Options	45
5.2	TLV-Parser und Serializer	45
5.2.1	Parsing-Algorithmus	45
5.2.2	Serialisierung	45
5.3	Option Checksum (OCS)	45
5.3.1	Berechnung	45
5.3.2	Validierung	45
5.4	Raw-Socket-Zugriff und Plattform-Spezifika	45
5.4.1	Socket-Erstellung und Paketempfang	45
5.4.2	Plattform-Spezifika unter Linux	45
5.4.3	Build-Konfiguration	46
6	Evaluation und Diskussion	47
6.1	Test- und Messkonzept	47
6.2	Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden . .	47
6.3	Soll-Ist-Abgleich mit RFC 9868	47
7	Fazit und Ausblick	48
7.1	Beantwortung der Forschungsfragen	48
7.2	Ausblick	48

7.3 Schlusswort	48
Literaturverzeichnis	49
Eidesstattliche Erklärung	53

Abbildungsverzeichnis

2.1	Der IPv4-Header nach RFC 791 [5]. Hervorgehoben sind die beiden Längenangaben, auf denen die Bestimmung der Surplus Area beruht; gestrichelte Felder sind nur vorhanden, wenn IHL über dem Minimalwert 5 liegt.	7
2.2	Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente	9
2.3	Der UDP-Header nach RFC 768 [1]	11
2.4	Der Pseudo-Header der UDP-Prüfsumme für IPv4 [1]	12
2.5	IP Transport Payload und Surplus Area eines Datagramms mit Total Length 60	15
2.6	Anatomie eines Datagramms mit UDP Options	16
2.7	Paritätsentscheid für das Pad-Byte vor dem OCS	17
2.8	Sende- und Empfangsweg eines UDP-Datagramms durch den Linux-Kernel (aus [4])	22
2.9	Die zwei Sendepfade durch den Linux-Kernel [4, 17]	22
2.10	Der Empfangspfad durch den Linux-Kernel [4, 17]	23
2.11	Der Lebenslauf eines Datagramms	25
3.1	Aufbau der Untersuchung und Zuordnung der Forschungsfragen . . .	26
3.2	Von den Normquellen zur Anforderungsmatrix	29
3.3	Rollen bei Erzeugung, Prüfung und Freigabe	30
3.4	Schwächen der Sprachmodelle und methodische Antworten	32
3.5	Prüfprozess im Sollzustand	34

Tabellenverzeichnis

3.1	Operationalisierung der Forschungsfragen	27
4.1	Vergleich der betrachteten Systemprogrammiersprachen anhand der für die RFC 9868-Implementierung maßgeblichen Kriterien	43

Abkürzungsverzeichnis

APC Alternate Payload Checksum.

AUTH Authentication Option.

DTLS Datagram Transport Layer Security.

EOL End of List.

FRAG Fragmentation Option.

MDS Maximum Datagram Size.

MRDS Maximum Reassembled Datagram Size.

NOP No Operation.

OCS Option Checksum.

PMTU Path Maximum Transmission Unit.

TIME Timestamp Option.

1. Einleitung

1.1 Motivation

Die ursprüngliche Spezifikation des User Datagram Protocol (UDP), RFC 768 aus dem Jahr 1980, umfasst 663 Wörter [1]. Die Erweiterung RFC 9868 wurde im Oktober 2025 veröffentlicht und führt unter dem Titel *Transport Options for UDP* erstmals einen allgemeinen Optionsmechanismus für UDP ein [2]. Zwischen beiden Dokumenten liegen 45 Jahre; die Klartextfassung des RFC 9868 ist mehr als 27-mal so umfangreich wie die des RFC 768.

Die Einfachheit von UDP ist dabei kein Mangel, sondern Entwurfsziel: Das Protokoll bietet lediglich Portnummern zur Demultiplexierung und eine Prüfsumme zur Fehlererkennung. Genau diese Einfachheit macht UDP zu einem der am häufigsten genutzten Transportprotokolle im Internet.

RFC 9868 greift Funktionen auf, die UDP ursprünglich der Anwendung oder einer darüberliegenden Schicht überließ: Transportfragmentierung, eine zusätzliche Nutzdatenprüfsumme sowie Optionen für Größen- und Zeitinformationen [2, Sec. 5, 11.3 und 11.8]. Für ihre Bereitstellung kommen grundsätzlich drei Wege in Betracht:

- Erweiterung innerhalb der Nutzdaten: Jede Anwendung definiert ein eigenes Rahmenformat in der UDP-Nutzlast.
- Ein neues Transportprotokoll: Alle gewünschten Mechanismen können so von Beginn an vorgesehen werden.
- Erweiterung des bestehenden Formats: UDP selbst erhält einen Optionsmechanismus, der für Bestandssysteme möglichst unsichtbar bleibt.

RFC 9868 wählt den dritten Weg (die Muster hinter diesen Wegen vergleicht Kapitel 2) und nutzt dafür die sogenannte *Surplus Area*: den möglichen Bereich zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms. Da das UDP-Length-Feld die Länge des UDP-Datagramms angibt und diese kleiner sein kann als die *IP Transport Payload*, also alles, was im IP-Datagramm auf den IP-Header folgt, entsteht ein bisher ungenutzter Bereich für Erweiterungen (Abschnitt 2.4). Auf dieser Grundlage definiert RFC 9868 ein erweiterbares Rahmenwerk für Transportoptionen.

Damit steht das minimalistische UDP vor seiner bislang größten Erweiterung. Nun stellt sich die Frage, die diese Arbeit als Leitfrage begleitet:

Wie lässt sich UDP um Transportoptionen erweitern, ohne seine grundlegenden Eigenschaften aufzugeben?

Die Arbeit untersucht diese Frage anhand einer Referenzimplementierung und auf ausgewählten realen Netzpfaden.

1.2 Problemstellung

Die Implementierung von UDP-Optionen gemäß RFC 9868 wirft drei Problemfelder auf, die diese Arbeit behandelt: die Beschaffenheit der Surplus Area selbst, die bewusste Unidirektionalität des Protokolls sowie die praktischen Implementierungsherausforderungen vom Zugriff aus dem Benutzerraum bis zum Verhalten von Zwischensystemen. Die folgenden Abschnitte behandeln sie in dieser Reihenfolge.

1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum

Anders als TCP besitzt der ursprüngliche UDP-Header kein explizites Optionsfeld. RFC 9868 platziert die Optionen deshalb außerhalb der durch das UDP-Length-Feld begrenzten Nutzdaten, aber noch innerhalb des IP-Datagramms. Damit bleibt der UDP-Header unverändert, die Implementierung muss jedoch zwei unterschiedliche Längengrenzen zuverlässig auswerten. Den genauen Aufbau der Surplus Area behandelt Abschnitt 2.4.

1.2.2 Unidirektionalität

RFC 9868 stellt explizit klar: „*UDP is unidirectional; UDP Options do not change that fact.*“ [2, Sec. 6].

Für Mechanismen, die Bidirektionalität voraussetzen, fordert RFC 9868 eine getrennte Spezifikation [2, Sec. 6]. RFC 9869 definiert einen solchen Einsatz von REQ und RES für die Pfad-MTU-Ermittlung [3, Sec. 3]. Diese Vorgabe schließt einen Datenaustausch in beiden Richtungen nicht aus. Sie bedeutet, dass die UDP-Optionsschicht keine Antwort selbstständig erzeugt. Nicht das Protokoll, sondern die Anwendung oder eine in ihrem Auftrag arbeitende Bibliothek löst eine Antwort aus.

1.2.3 Herausforderungen

Die zentrale Herausforderung bei der Implementierung von UDP Optionen liegt im Zugriff auf die Surplus Area. Existierende Betriebssystem-Stacks liefern über Standard-Socket-APIs ausschließlich die durch das UDP-Length-Feld begrenzten Nutzdaten an die Anwendungsschicht; die Surplus Area wird stillschweigend verworfen

[2, Sec. 18]. Den Weg eines UDP-Pakets durch den Linux-Kernel bis zu dieser Socket-Schnittstelle dokumentieren Stephan und Wüstrich [4].

Eine weitere Herausforderung stellt die Kompatibilität mit Zwischensystemen dar. RFC 9868 berichtet Interoperabilitätstests, in denen Pakete mit Surplus Area Geräte zur Netzadressübersetzung (NAT, *Network Address Translation*) passierten, nennt aber auch ein Intrusion Detection System, das die Diskrepanz zwischen UDP-Length und IP-Length in der Standardeinstellung als Angriff meldet [2, Sec. 18]. Ob reale Netzpfade solche Pakete tatsächlich durchlassen, ist eine empirische Frage dieser Arbeit (FF2).

1.3 Zielsetzung

Die Problemstellung verdichtet sich zu zwei getrennten Hürden: Ein UDP-Options-Paket muss normgerecht erzeugt und empfangen werden, und seine Surplus Area muss die untersuchten Netzpfade unverändert überstehen.

Eine normgerechte Implementierung sagt nichts darüber aus, ob Zwischensysteme Datagramme mit Surplus Area unverändert passieren lassen. Ebenso nützt ein durchlässiger Netzpfad wenig, wenn Erzeugung und Auswertung der Optionen fehlerhaft sind. Aus dieser Trennung ergeben sich zwei Forschungsfragen:

- **FF1:** Welche Anforderungen des RFC 9868 lassen sich unter Linux und IPv4 mit einer Raw-Socket-Implementierung im Benutzerraum vollständig, teilweise oder nicht erfüllen?
- **FF2:** In welchem Umfang bleibt die *Surplus Area* auf den untersuchten realen Netzpfeiden bis zur Gegenstelle unverändert erhalten?

Das Ziel dieser Arbeit ist es, beide Fragen zu beantworten. Dazu entsteht zunächst eine an RFC 9868 ausgerichtete und systematisch auf Konformität geprüfte Bibliothek für UDP-Transportoptionen in Rust. Die auf Linux und IPv4 begrenzte Implementierung arbeitet im Benutzerraum mit Raw Sockets. Sie umfasst Parser und Serialisierer für UDP-Optionen nach dem TLV-Schema (*Type-Length-Value*), die Validierung der Optionsprüfsumme (*Option Checksum*, OCS) gemäß Abschnitt 9 des RFC 9868 sowie die verpflichtend zu unterstützenden Optionen EOL, NOP, APC, FRAG, MDS, MRDS, REQ und RES [2, Sec. 10].

Die Verifikation erfolgt mehrstufig: Komponententests prüfen die einzelnen Bestandteile, Integrationstests den Loopback-Pfad, und für reine Regeln wird ergänzend eine Lean-Spezifikation gepflegt. Lean dient als formale Gegenprobe für Prüfsummenarithmetik sowie Längen- und Anordnungsregeln; das Verhalten von Raw Sockets,

Betriebssystemen und Zwischensystemen bleibt Gegenstand empirischer Prüfungen. Dieses Prüfprogramm schafft die Grundlage für die Beantwortung von FF1.

Für FF2 vergleicht die Evaluation gewöhnliche UDP-Datagramme mit gleich großen Datagrammen, die eine Surplus Area enthalten, auf ausgewählten realen Netzpfeilen. Sie unterscheidet zwischen unveränderter Zustellung, Entfernung der Surplus Area und Paketverlust. Die Schlussfolgerungen bleiben auf die beobachteten Pfade begrenzt.

1.4 Abgrenzung und Umfang

Diese Arbeit setzt einen bewussten Rahmen. Sie beschränkt sich auf eine Umsetzung des RFC 9868 im Linux-Benutzerraum für IPv4. Gegenstand sind das TLV-Rahmenwerk, die OCS sowie die verpflichtend zu unterstützenden Optionen. Die Beschränkung auf den Benutzerraum ist eine vorab gesetzte Rahmenbedingung der Aufgabenstellung. Abschnitt 4.4 begründet die dafür gewählte Umsetzung mit Raw Sockets gegenüber kernelnahen Verfahren.

Außerhalb des Umfangs liegt der REQ/RES-Anwendungsfall zur Pfad-MTU-Ermittlung, den RFC 9869 definiert [3]. Die Implementierung unterstützt nur die in RFC 9868 festgelegten Formate von REQ und RES. Ebenfalls nicht implementiert wird die TIME-Option. RFC 9868 beschreibt zwar ihr Format und den Anwendungszweck, die Schätzung der Paketumlaufzeit (*Round-Trip Time*, RTT), legt aber kein nutzbares Protokoll fest. Die Kennungen für AUTH, UCMP und UENC sind in RFC 9868 lediglich reserviert und werden daher ebenfalls nicht implementiert (AUTH [2, Sec. 11.9], UCMP [2, Sec. 12.1], UENC [2, Sec. 12.2]). Kernel-Module werden nicht entwickelt.

IPv6 wird weder implementiert noch evaluiert. Die hier untersuchten Mechanismen des RFC 9868 (*Surplus Area*, OCS, TLV-Optionen und FRAG) lassen sich vollständig über IPv4 zeigen. Die Implementierung und ihre Konformitätsprüfung beziehen sich daher auf IPv4.

1.5 Aufbau der Arbeit

Nach dieser Einleitung folgen sechs Kapitel, die drei Blöcke bilden:

- **Grundlagen und Methodik:** Kapitel 2 führt die technischen Grundlagen ein: das Internet Protocol, UDP, Erweiterungsmuster von Transportprotokollen sowie RFC 9868 mit der *Surplus Area* und seinen Entwurfsprinzipien. Kapitel 3 legt die Forschungsmethodik offen, bevor mit ihr gearbeitet wird: das Forschungsdesign, die RFC-Auswertungsmethode und der Einsatz von KI.

- **Anwendungsteil:** Kapitel 4 überführt RFC 9868 mit dieser Methode in ein prüfbares Anforderungsmodell und begründet die Technologieauswahl. Kapitel 5 führt Entwurf und Implementierung der Bibliothek komponentenweise zusammen. Kapitel 6 berichtet die Ergebnisse zu beiden Forschungsfragen sowie eine deskriptive Auswertung der KI-gestützten Arbeitsweise.
- **Schluss:** Kapitel 7 wiederholt die Forschungsfragen, beantwortet sie einzeln und zieht die Bilanz zur Leitfrage.

Diese lineare Kapiteelfolge ist eine rekonstruktive Darstellungsstruktur und kein Abbild des tatsächlichen Arbeitsprozesses. Die Umsetzung verlief iterativ und agenten-gestützt, sodass Entwurf, Implementierung und Test einander fortlaufend bedingen.

1.6 KI-Einsatz

Diese Arbeit ist unter durchgehendem Einsatz künstlicher Intelligenz (KI) entstanden. Der Einsatz betrifft Text und Software. Auf der Textseite erstellten KI-Werkzeuge Entwürfe, überarbeiteten Formulierungen, übernahmen die LaTeX-Formatierung, setzten die Abbildungen als TikZ-Grafiken um und bedienten die LaTeX-Werkzeugkette. Zudem prüften sie den Text wiederholt gegen den Primärtext des RFC 9868. Auf der Softwareseite unterstützten sie Planung, Implementierung, Tests und Verifikation der Bibliothek. Der Autor entschied über die Übernahme der Ergebnisse und verantwortet sie.

Zum Einsatz kommen zwei KI-Werkzeuge (harness), die ein Sprachmodell mit Dateizugriff und Kommandoausführung verbinden: Claude Code von Anthropic und Codex von OpenAI. Ihre Rollenverteilung, die bekannten Schwächen dieser Arbeitsweise und die Prüfkette vor der Übernahme eines Ergebnisses behandelt Kapitel 3.

2. Grundlagen

Dieses Kapitel stellt die Grundlagen bereit, die zum Verständnis von RFC 9868 und zur Beurteilung der späteren Implementierung und Messungen nötig sind: IPv4 und UDP als die beiden beteiligten Protokolle (Abschnitt 2.1, Abschnitt 2.2), die Erweiterungsmuster im Vergleich (Abschnitt 2.3), die Surplus Area samt Entwurfsprinzipien (Abschnitt 2.4) sowie Betriebssystem und Netzpfad der Messungen (Abschnitt 2.5).

2.1 IPv4

Das Internet Protocol (IP) vermittelt Datagramme über die Grenzen einzelner Netze hinweg zwischen Endsystemen und bildet damit die Vermittlungsschicht, auf der UDP aufsetzt [5]. Dass diese Arbeit eine Schicht unterhalb ihres eigentlichen Gegenstands beginnt, hat einen konkreten Grund: Die *Surplus Area*, in der RFC 9868 seine Optionen ablegt, ist allein durch das Zusammenspiel beider Schichten definiert. Das UDP-Length-Feld markiert das Ende der UDP-Nutzdaten; wo das umgebende IP-Datagramm endet, ergibt sich dagegen ausschließlich aus dem IP-Header. Ohne die Längenangaben von IPv4 lässt sich daher weder feststellen, ob eine Surplus Area vorhanden ist, noch wie groß sie ausfällt [2, Sec. 7].

Dabei genügt der Blick auf die Version 4 des Protokolls. RFC 9868 definiert die Surplus Area für IPv4 und IPv6 gleichermaßen, und sämtliche Optionsmechanismen sind in beiden Fällen identisch; die Versionen unterscheiden sich nur darin, wie das Ende der Nutzlast bestimmt wird [2, Sec. 7]. Bei IPv4 reichen dafür zwei Felder des Headers, während bei IPv6 die Kette der Extension Header durchlaufen werden muss [6]. Da somit kein Mechanismus des Standards von der IP-Version abhängt, lässt er sich vollständig an IPv4 zeigen. Implementierung und Evaluation dieser Arbeit sind entsprechend auf IPv4 begrenzt (Abschnitt 1.4).

Auch IPv4 selbst wird in diesem Unterkapitel nicht vollständig erklärt und dargestellt. Dieser Abschnitt behandelt ausschließlich die Themen, Header-Elemente und Eigenschaften, die RFC 9868 und die spätere Implementierung tatsächlich berühren: die beiden Längenangaben des Headers, das Protokollfeld, die Adressen, die Header-Prüfsumme, das Fragmentierungsverhalten sowie das Feld `Time to Live` als Messgröße der Evaluation.

Den Ausgangspunkt bildet der Header; seinen Aufbau zeigt Abbildung 2.1 [5].

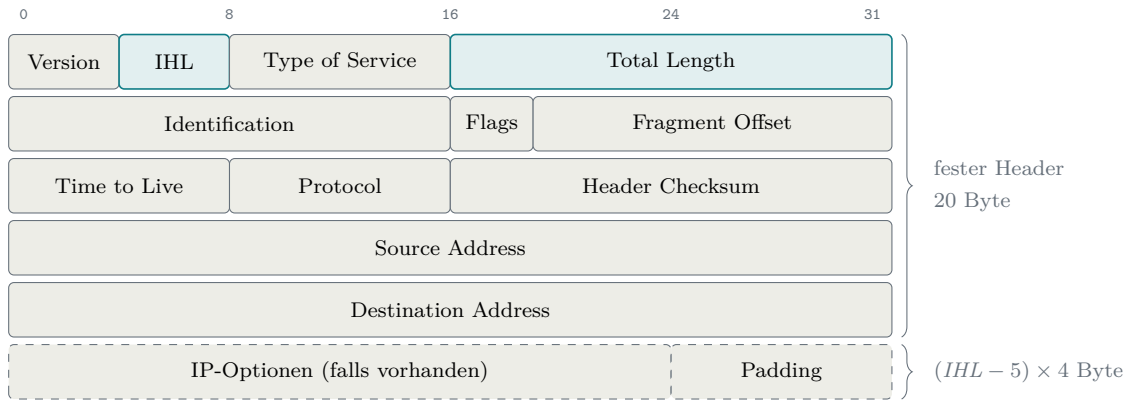


Abbildung 2.1: Der IPv4-Header nach RFC 791 [5]. Hervorgehoben sind die beiden Längenangaben, auf denen die Bestimmung der Surplus Area beruht; gestrichelte Felder sind nur vorhanden, wenn IHL über dem Minimalwert 5 liegt.

Im Zentrum stehen die beiden Längenangaben der ersten Zeile. **Total Length** gibt die Gesamtlänge des Datagramms in Byte an, Header und Nutzlast zusammen; als 16-Bit-Feld erlaubt es höchstens 65 535 Byte.

Die zweite Längenangabe, die **IHL** (Internet Header Length), beziffert die Länge des Headers. Sie zählt dabei nicht in Byte, sondern in 32-Bit-Wörtern zu je 4 Byte. Der Grund liegt in der Breite des Feldes: Mit seinen 4 Bit kann es höchstens den Wert 15 darstellen. In Byte gezählt würde das nicht einmal für den festen Headeranteil von 20 Byte reichen; in 32-Bit-Wörtern gezählt beschreibt derselbe Wertebereich dagegen Header bis 60 Byte [5]. Die Headerlänge ergibt sich daher als

$$\text{Headerlänge} = IHL \times 4 \text{ Byte.} \quad (2.1)$$

Der Minimalwert 5 entspricht dem optionslosen 20-Byte-Header, der Maximalwert 15 einem Header von 60 Byte. Eine IPv4-Headerlänge ist damit stets ein Vielfaches von 4; diese unscheinbare Eigenschaft kehrt bei der Ausrichtung der UDP Options wieder.

Zwischen festem Header und Nutzlast können zudem IP-Optionen liegen. Sie stellen Steuerfunktionen für Sonderfälle bereit, die die gewöhnliche Kommunikation nicht braucht [5] und dienen als zusätzliche Felder zur Erweiterung der IP-Header: **Record Route** etwa sammelt die Adressen der durchlaufenen Router ein, **Internet Timestamp** deren Zeitstempel. In der Praxis werden solche Optionen kaum genutzt: Optionstragende Pakete fallen aus dem schnellen Weiterleitungspfad der Router heraus, gelten deshalb als Überlastrisiko und werden verbreitet gefiltert [7, Sec. 1 und 3.1]; zudem fasst der 40-Byte-Optionsraum höchstens neun Adressen und damit viele reale Pfade nicht. Abschließend gilt, dass erst die IHL und keine Konstante den Beginn der Nutzlast festlegt.

Aus den beiden Längenangaben **Total Length** und **IHL** ergibt sich die Größe,

auf die sich RFC 9868 durchgängig bezieht: Die Nutzlast eines IPv4-Datagramms beginnt nach `IHL × 4 Byte` und endet bei `Total Length`. Für ein UDP-Datagramm fordert RFC 9868 deshalb [2, Sec. 7]¹

$$UDP\ Length \leq Total\ Length - Headerlänge. \quad (2.2)$$

Die rechte Seite der Ungleichung ist die Länge dieser Nutzlast, also der Platz hinter dem IP-Header. Die linke Seite ist die Länge, die das UDP-Datagramm für sich beansprucht. Die Bedingung verlangt damit, dass das UDP-Datagramm vollständig in die Nutzlast des IP-Datagramms passt; UDP darf nie mehr Platz beanspruchen, als das IP-Datagramm hergibt. Bei Gleichheit füllt das UDP-Datagramm die Nutzlast exakt aus; so arbeiten klassische Sender. Bleibt `UDP Length` dagegen zurück, ist die Differenz genau der Raum, den Abschnitt 2.4 als Surplus Area einführt.

Von den übrigen Feldern braucht die Arbeit nur wenige. `Protocol` benennt das Transportprotokoll der Nutzlast; der Wert 17 steht für UDP. Die `Header Checksum` sichert ausschließlich den Header, nicht die Nutzlast; deren Fehlererkennung bleibt der UDP-Prüfsumme überlassen (Abschnitt 2.2.1). Quell- und Zieladresse geben Absender und Empfänger des Datagramms an. Sie werden später noch einmal wichtig: Die UDP-Prüfsumme bezieht beide Adressen über einen sogenannten Pseudo-Header in ihre Berechnung ein. `Version` und `Type of Service` berühren den Optionsmechanismus nicht und werden nicht weiter vertieft.

Auch `Time to Live` spielt für die Optionen keine Rolle, kehrt in der Evaluation aber als Messgröße wieder: Jeder Router verringert den Wert beim Weiterleiten um eins und verwirft das Datagramm, wenn null erreicht ist [5]. Aus der Differenz zwischen gesendetem und empfangenem Wert liest Kapitel 6 daher die Zahl der durchlaufenen IPv4-Router ab und prüft damit die Stabilität der Messpfade. Die zweite Zeile des Headers schließlich (`Identification, Flags, Fragment Offset`) gehört vollständig zur Fragmentierung, der sich der Rest dieses Abschnitts widmet.

Wie groß ein Datagramm praktisch werden kann, entscheidet allerdings nicht das 16-Bit-Längenfeld, sondern der Übertragungsweg. Jede Technik der Sicherungsschicht begrenzt die Nutzlast eines einzelnen Rahmens; diese Obergrenze heißt Maximum Transmission Unit (MTU) und liegt für Ethernet üblicherweise bei 1500 Byte. Streng genommen ist die MTU also eine Eigenschaft der Sicherungsschicht. Ihre Folgen trägt jedoch die Vermittlungsschicht, denn ein Datagramm durchquert auf dem Weg zum Ziel mehrere Netze mit möglicherweise unterschiedlichen MTUs, und IP muss festlegen, was mit einem Datagramm geschieht, das nicht in den nächsten Rahmen

¹ In Ausnahmefällen liegen zwischen IPv4- und UDP-Header noch Shim-Header, etwa bei IPsec oder IPComp; das Feld `Protocol` zeigt dann nicht UDP an, und die Obergrenze verringert sich zusätzlich um die Gesamtlänge dieser Shim-Header [2, Sec. 7]. Diese Arbeit betrachtet solche Ketten nicht weiter.

passt [5].

IPv4 beantwortet das mit Fragmentierung. Ist ein Datagramm größer als die MTU und das Flag **DF** (Don't Fragment) nicht gesetzt, zerlegt die IP-Schicht es in mehrere Fragmente. Das übernimmt nicht nur der Absender: Bei IPv4 darf auch jeder Router unterwegs ein Datagramm zerlegen, das nicht in sein nächstes Netz passt [5]. Jedes Fragment erhält einen eigenen IP-Header, übernimmt die **Identification** des Originals, trägt in **Fragment Offset** seine Position in Einheiten von 8 Byte und setzt das Flag **MF** (More Fragments), das nur beim letzten Fragment fehlt. Der Empfänger sammelt zusammengehörige Fragmente anhand von Quelladresse, Zieladresse, Protokoll und **Identification** und setzt das Datagramm vor der Übergabe an die Transportschicht wieder zusammen [5]. Ist **DF** dagegen gesetzt, verwirft ein Router das zu große Datagramm und meldet dies dem Absender per ICMP; auf dieser Rückmeldung baut die Path MTU Discovery auf, mit der ein Sender die kleinste MTU seines Pfades ermittelt [8, Sec. 3.2].

Für diese Arbeit ist an der Fragmentierung vor allem eine Konsequenz bedeutsam: Der UDP-Header gehört aus Sicht von IP zur Nutzlast und landet deshalb vollständig im ersten Fragment. Was das für die übrigen Fragmente bedeutet, zeigt Abbildung 2.2 am Beispiel eines in drei Teile zerlegten Datagramms.

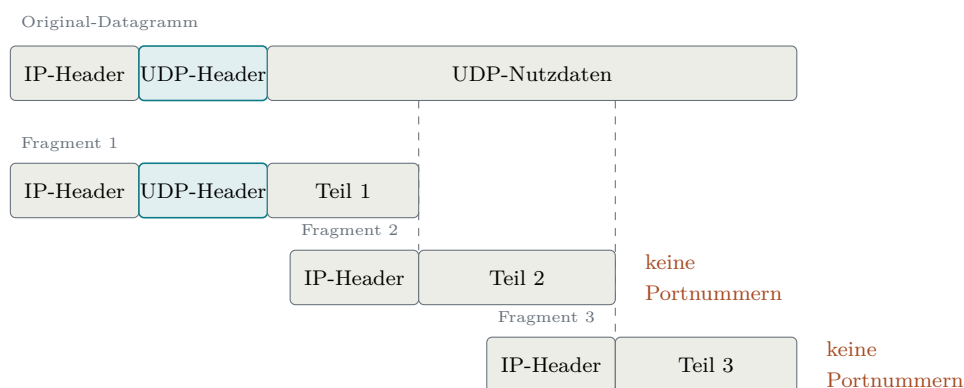


Abbildung 2.2: Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente

Aus Abbildung 2.2 geht hervor, dass nur das erste Fragment den UDP-Header enthält: Alle Folgefragmente tragen keine Portnummern. Zwischensysteme wie NATs und Firewalls, die Pakete anhand vollständiger Header zuordnen oder prüfen, können solche Fragmente nicht einordnen und verwerfen sie teils pauschal; zudem macht der Verlust eines einzelnen Fragments das gesamte Datagramm unbrauchbar. RFC 8085 wertet beides als grundsätzliches Problem der IP-Fragmentierung, das jedes Transportprotokoll trifft, und rät UDP-Anwendungen deshalb, IP-Fragmentierung zu vermeiden [8, Sec. 3.2]. Dass der Rat gerade an UDP geht, hat einen Grund: Während TCP seinen Datenstrom selbst in passende Segmente zerlegt und der IP-Fragmentierung so meist entgeht [9], erzeugt UDP je Nachricht genau ein Datagramm (Abschnitt 2.2.2);

ein zu großes Datagramm kann bislang nur die IP-Schicht zerteilen. Genau diese Lücke schließt die FRAG-Option von RFC 9868: Sie verlagert die Fragmentierung auf die Transportschicht und wiederholt die Portnummern in jedem Fragment [2, Sec. 11.4]; den Verlust einzelner Fragmente behebt allerdings auch FRAG nicht. Darauf kommt Kapitel 4 zurück.

Der folgende Abschnitt wendet sich dem Protokoll zu, das RFC 9868 tatsächlich erweitert: UDP.

2.2 Das User Datagram Protocol (UDP)

Das User Datagram Protocol (UDP) ist neben dem Transmission Control Protocol (TCP) eines der beiden zentralen Transportprotokolle der Transportschicht. Im Gegensatz zu TCP verfolgt UDP ein bewusst minimalistisches Design: Es sendet Datagramme ohne vorherigen Handshake oder Zustandsabgleich mit dem Empfänger, bietet keine Garantien für Zustellung, Reihenfolge oder Duplikaterkennung und verzichtet auf jeglichen Flusskontroll- oder Staukontrollmechanismus [1]. Mit der Protokollnummer 17 im IP-Header registriert, wurde UDP in RFC 768 spezifiziert und trägt die Bezeichnung STD 6 als Internet-Standard. Bemerkenswert ist, dass RFC 768 seit seiner Veröffentlichung im August 1980 über 45 Jahre hinweg keine formelle Aktualisierung erfahren hat. Erst RFC 9868 trägt den Header-Eintrag **Updates: 768** und erweitert damit den ursprünglichen UDP-Standard erstmalig direkt [2].

Historisch geht UDP auf die Aufspaltung des ursprünglichen *Transmission Control Program* zurück, das Cerf, Dalal und Sunshine Ende 1974 noch als monolithische Einheit aus Netzwerk- und Transportfunktionen beschrieben [10]. Mit deren Trennung in IP und TCP entstand der Bedarf nach einer schlanken, transaktionsorientierten Alternative ohne den Overhead von TCP. Während der UDP-Kern über die folgenden Jahrzehnte unverändert blieb und nur Begleitdokumente Randfragen klärten, ließ erst die wachsende Bedeutung UDP-basierter Protokolle wie QUIC [11] und HTTP/3 [12] mit RFC 9868 die erste direkte Erweiterung des Standards entstehen.

2.2.1 Protokollstruktur

Ein UDP-Datagramm besteht aus einem Header mit fester Länge von 8 Byte und den darauf folgenden Nutzdaten; den Aufbau des Headers zeigt Abbildung 2.3 [1].

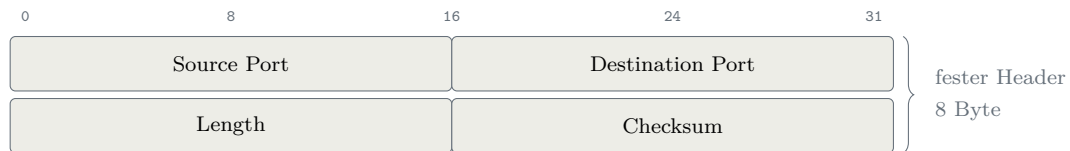


Abbildung 2.3: Der UDP-Header nach RFC 768 [1]

Aus Abbildung 2.3 geht hervor, dass der gesamte Header aus genau vier Feldern zu je 16 Bit besteht; unter ihnen das Längenfeld, für das Abschnitt 2.1 mit Gleichung 2.2 bereits die Obergrenze aufgestellt hat. Die vier Felder im Einzelnen:

- **Source Port:** Die Portnummer des sendenden Prozesses. Dieses Feld ist optional; wird es nicht benötigt, wird es auf null gesetzt [1]. In der Praxis nutzen Anwendungen den *Source Port* zur Identifikation des Absenders, damit der Empfänger Antworten an den korrekten Port zurücksenden kann.
- **Destination Port:** Die Portnummer des Zielprozesses auf dem empfangenden Host. Zusammen mit der IP-Zieladresse ermöglicht dieses Feld die *Demultiplexierung* eingehender Datagramme an die korrekte Anwendung.
- **Length:** Die Gesamtlänge des UDP-Datagramms in Byte, bestehend aus Header und Nutzdaten. Der Minimalwert beträgt 8, was einem Datagramm ohne Nutzdaten entspricht [1]. Da das Feld 16 Bit breit ist, ergibt sich eine theoretische Maximalgröße von 65 535 Byte.
- **Checksum:** Eine Prüfsumme zur Fehlererkennung über einen Pseudo-Header, den UDP-Header und die Nutzdaten; ihr Verfahren und ihre Sonderwerte erläutert der folgende Absatz.

Prüfsumme. Die Prüfsumme dient der Fehlererkennung: Mit ihr erkennt der Empfänger, ob einzelne Bits des Datagramms auf dem Weg verändert wurden. Diese Prüfung auf der Transportschicht ist nötig, weil nicht jede Teilstrecke eines Pfades eigene Fehlererkennung bietet und Bits auch im Speicher eines Routers kippen können [13]. Dennoch ist die Prüfsumme optional. Für die Berechnung summiert der Sender Pseudo-Header, UDP-Header und Nutzdaten als 16-Bit-Wörter im Einerkomplement auf (ein Übertrag wird am niederwertigen Ende wieder hinzugezählt); das invertierte Ergebnis bildet den Feldwert [1, 14]. Der Empfänger summiert dieselben Wörter einschließlich des Prüfsummenfelds und erwartet lauter Einsen, also 0xFFFF; jedes andere Ergebnis zeigt einen Übertragungsfehler an [14].

Dabei hilft eine Eigenheit des Einerkomplements: Es kennt zwei Darstellungen der Null, 0x0000 und 0xFFFF. Diese Doppelrolle nutzt UDP für die Sonderwerte des Feldes: eine übertragene 0x0000 bedeutet, dass der Sender keine berechnet hat.

Ergibt die Berechnung selbst den Wert null, wird stattdessen `0xFFFF` übertragen. Dadurch bleibt im Einerkomplement die Zahl Null erhalten, und der Sonderwert bleibt eindeutig [1]. Dabei ist zu beachten, dass UDP Fehler nur erkennen und nicht beheben kann; ein beschädigtes Datagramm wird typischerweise verworfen [13].

Pseudo-Header. Die UDP-Prüfsumme schützt nicht nur die Transportschichtdaten, sondern bezieht über einen *Pseudo-Header* auch Informationen der Vermittlungsschicht ein. Dieser Pseudo-Header ist kein Teil des Datagramms und steht an keiner Stelle im Paket: Sender und Empfänger bauen ihn jeweils nur für die Berechnung im Speicher auf und stellen ihn dem UDP-Header dabei gedanklich voran [1]. Dieses Konzept stellt sicher, dass ein Datagramm, das durch einen IP-Fehler an den falschen Host oder das falsche Protokoll zugestellt wird, erkannt und verworfen werden kann. Seinen Aufbau zeigt Abbildung 2.4 [1].

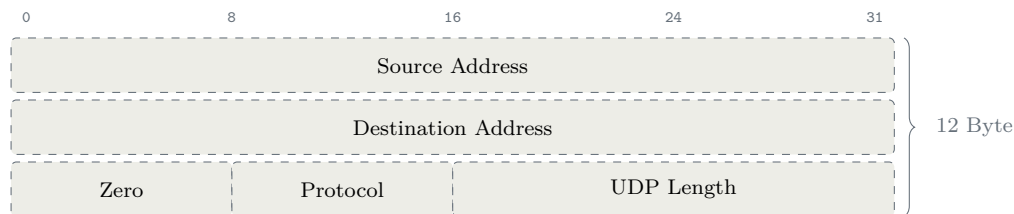


Abbildung 2.4: Der Pseudo-Header der UDP-Prüfsumme für IPv4 [1]

Aus Abbildung 2.4 geht hervor, dass der Pseudo-Header ausschließlich aus Werten besteht, die beiden Seiten ohnehin vorliegen: den beiden Adressen und dem Feld `Protocol` aus dem IP-Header (Abschnitt 2.1), einem Nullbyte als Füllung sowie einer Wiederholung des UDP-Längenfelds. Nur deshalb können Sender und Empfänger unabhängig voneinander dieselben zwölf Byte zusammensetzen; die gestrichelte Darstellung kennzeichnet, dass keines dieser Felder je übertragen wird [1].

Ein letzter Blick gilt dem Feld `Zero`: Es trägt keine Information, sondern stellt die Ausrichtung her. Die Prüfsummenarithmetik arbeitet ausschließlich auf 16-Bit-Wörtern [14], das Protokollfeld aus dem IP-Header ist aber nur 8 Bit breit; das vorangestellte Nullbyte ergänzt es zu einem vollen Wort. Nach demselben Prinzip werden auch Nutzdaten ungerader Länge für die Berechnung am Ende mit einem Nulloktett aufgefüllt [1]. Durch ein anderes Feld aus dem IP-Header, etwa `Time to Live`, lässt sich das Nullbyte dagegen nicht ersetzen: Jeder Router verringert diesen Wert beim Weiterleiten (Abschnitt 2.1); der Empfänger würde also mit einem anderen Wert rechnen als der Sender, und die Prüfung ginge selbst bei fehlerfreier Übertragung nie auf. In den Pseudo-Header gehören nur Werte, die unterwegs unverändert bleiben; ein festes Nullbyte erfüllt das von selbst.

2.2.2 Eigenschaften

Die in Abschnitt 2.2.1 beschriebene Header-Struktur verdeutlicht bereits das minimalistische Design von UDP. Aus diesem Designziel leiten sich eine Reihe funktionaler Eigenschaften ab, die maßgeblich beeinflussen, für welche Anwendungsfälle das Protokoll geeignet ist.

Nachrichtenorientierung. UDP ist ein *nachrichtenorientiertes* Protokoll: Jede Sendeoperation erzeugt genau ein Datagramm, und der Empfänger erhält jede Nachricht als atomare Einheit. TCP arbeitet dagegen *stromorientiert*: Es bietet der Anwendung einen fortlaufenden Bytestrom, den es für die Übertragung selbst in Segmente zerlegt [9, Sec. 2.2]. Sendet eine Anwendung etwa nacheinander "hello", "world" und "!", stellt UDP drei Datagramme mit genau diesen Nachrichten zu; ein TCP-Empfänger kann denselben Strom auch als "hellowo" und "rld!" erhalten [9, Sec. 3.7]. Eine Anwendung, die über TCP einzelne Nachrichten austauschen will, muss deren Grenzen deshalb selbst kennzeichnen, etwa durch Längfelder; bei UDP entfällt das, weil jedes Datagramm genau eine Nachricht trägt.

Zustandslosigkeit. UDP verwaltet keinen protokollspezifischen Zustand für einzelne Kommunikationsbeziehungen und hält keinerlei Datenstrukturen wie Sequenznummern, Fenstergrößen oder Retransmission-Timer vor [2, Sec. 6].

Minimale Latenz. Da UDP verbindungslos arbeitet, entfällt jeglicher Handshake vor der Datenübertragung. Die erste Nachricht kann unmittelbar gesendet werden, ohne dass ein vorheriger Verbindungsaufbau abgewartet werden muss [8, Sec. 1].

Unabhängigkeit der Datagramme. Jedes UDP-Datagramm ist eine eigenständige, von allen anderen Datagrammen unabhängige Einheit. Der Verlust eines Datagramms hat daher keine Auswirkung auf die Verarbeitung nachfolgender Datagramme. Diese Eigenschaft vermeidet das sogenannte *Head-of-Line-Blocking* (HoL-Blocking), bei dem verlorene Pakete die Auslieferung nachfolgender Daten verzögern. Diese Eigenschaft von UDP war eine der Motivationen für die Entwicklung von QUIC: QUIC nutzt UDP als Transportschicht und realisiert auf Anwendungsebene ein Stream-Multiplexing, bei dem der Verlust eines Pakets nur den betroffenen Stream blockiert, nicht jedoch andere parallele Streams [11, Sec. 1].

Diese Eigenschaften machen UDP zu einem attraktiven Transportprotokoll für latenzempfindliche oder verlusttolerante Anwendungen. Allerdings fehlt UDP jede Form der protokolleigenen Erweiterbarkeit. Genau diese Lücke adressiert RFC 9868 durch die Nutzung der *Surplus Area* (siehe Abschnitt 2.4), um Optionen zu transportieren, ohne den bestehenden UDP-Header zu verändern.

2.3 Erweiterungsmuster im Vergleich

Nun stellt sich die Frage, wie ein Protokoll nachträglich erweitert werden kann, dessen Header weder freie Bits noch ein Versionsfeld bietet. Das klassische Muster ist ein Optionsbereich zwischen festem Header und Nutzdaten: Ein Offsetfeld gibt an, wo die Nutzdaten beginnen, und der Raum dazwischen steht für Optionen zur Verfügung. TCP arbeitet genau so. Sein Feld `Data Offset` zählt wie die `IHL` in 32-Bit-Wörtern und ist 4 Bit breit; vom 60-Byte-Maximum bleiben nach dem festen 20-Byte-Header höchstens 40 Byte für Optionen wie Maximum Segment Size, Window Scale oder Timestamps [9]. Dasselbe Muster kennt auch IPv4: Die IP-Optionen aus Abschnitt 2.1 liegen genauso zwischen festem Header und Nutzlast. TCP und IPv4 teilen dabei sogar das Füllmuster aus den Ein-Byte-Codes `NOP` (No Operation) und `EOL` (End of Option List).

Für UDP steht dieser Weg nicht offen. Sein Header besteht aus genau vier 16-Bit-Feldern (Abschnitt 2.2.1); es gibt kein Offsetfeld, kein Versionsfeld und kein reserviertes Bit, das einen Optionsbereich ankündigen könnte. Jede Änderung am Headerformat würde sämtliche bestehenden Implementierungen brechen und scheidet damit aus.

Ein erster Ausweg verzichtet auf jede Änderung am Protokoll und legt die Optionen in die Nutzdaten selbst. Diesen Weg gehen die Erweiterungen von Teredo, einem Tunnelprotokoll, das IPv6-Pakete in UDP-Datagramme verpackt und so Hosts hinter NATs den Zugang zum IPv6-Internet öffnet [15]. Ihre Zusatzfelder stehen als *Trailer* hinter dem getunnelten Paket, noch innerhalb der UDP-Nutzdaten: Was ein Header vor den Daten ist, ist ein Trailer hinter ihnen. Weil das getunnelte Paket sein Ende selbst markiert, liest der Empfänger alles dahinter als Trailer [16]. Die Stärke dieses Ansatzes liegt auf der Hand: UDP und IP bleiben unangetastet, das Datagramm ist nach außen ein gewöhnliches Datagramm, und ein Empfänger, der nur das innere Paket auswertet, stoppt an dessen Ende und stört sich nicht an den Trailern. Die Schwäche liegt in der Voraussetzung: Das Verfahren funktioniert nur, wenn die Nutzdaten ein eigenes Längenfeld mitbringen, das ihr Ende markiert. Beliebige UDP-Anwendungen erfüllen das nicht; ihnen bliebe nur, Zusatzfelder per Konvention innerhalb der Nutzdaten von den Anwendungsdaten zu unterscheiden. Solche In-Band-Kodierung verwirft RFC 9868 jedoch, weil Altempfänger die Konvention nicht kennen und die Zusatzfelder als Nutzdaten lesen [2, Sec. 4].

Den Weg zur Surplus Area öffnet schließlich eine Eigenheit, die UDP von TCP unterscheidet: UDP trägt eine eigene Längenangabe. TCP leitet die Länge seiner Nutzlast aus der IP-Datagrammlänge ab; UDP wiederholt sie im eigenen Header [2, Sec. 4]. Damit beschreiben zwei unabhängige Angaben redundant dasselbe Ende der Nutzdaten: das UDP-Length-Feld und die IP-Längenangaben aus Abschnitt 2.1. Genau diese

Redundanz überträgt den Trailer-Gedanken auf beliebige UDP-Anwendungen, denn das Längenfeld, das das Ende der Daten markiert, muss nun nicht mehr in den Nutzdaten stecken: UDP selbst bringt es mit. Bleibt `UDP Length` hinter der Nutzlast des IP-Datagramms zurück, entsteht hinter den Nutzdaten Platz, den ein Altempfänger nie zu Gesicht bekommt, weil er nur bis `UDP Length` liest. Mit den Teredo-Trailern hat dieser Bereich nichts zu tun; beide Mechanismen bleiben deshalb kompatibel [2, Sec. 4]. Warum RFC 768 das Längenfeld überhaupt vorsieht, ist bemerkenswerterweise ungeklärt: Die kursierenden Erklärungen (mehrere UDP-Pakete je IP-Datagramm; Abgrenzung der Nutzdaten von erzwungenem Null-Padding) hält RFC 9868 für unvereinbar mit früheren UDP-Entwürfen [2, Sec. 4]. Ein Feld, dessen Zweck sich nie klärte, wird damit 45 Jahre nach seiner Definition zum Erweiterungspunkt des Protokolls.

Von den drei Mustern ist für UDP damit allein der Trailer-Weg gangbar: Header-Optionen scheitern am unveränderlichen Headerformat, die In-Band-Kodierung an den Altempfängern, denen die Konvention unbekannt ist. Der Trailer bleibt für Altempfänger unsichtbar und erbt zugleich keine feste Obergrenze für den Optionsraum, denn seine Grenze ist das Ende des IP-Datagramms. Wie dieser Bereich im Einzelnen aufgebaut ist, zeigt der folgende Abschnitt.

2.4 Die Surplus Area

Da das `UDP-Length`-Feld die Länge des UDP-Datagramms (Header und Nutzdaten) angibt und diese kleiner sein kann als die Nutzlast, die das IP-Datagramm laut seinen Längenangaben transportiert (Abschnitt 2.1), entsteht zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms ein bislang ungenutzter Bereich. RFC 9868 nennt die gesamte Nutzlast hinter dem IP-Header *IP Transport Payload* und ihren ungenutzten Teil hinter den UDP-Nutzdaten *Surplus Area*; dort liegen die UDP Options. Die Lage beider Bereiche zeigt Abbildung 2.5 an einem Beispieldatagramm mit `Total Length` 60, `IHL` 5 und `UDP Length` 32 [2, Sec. 7]:

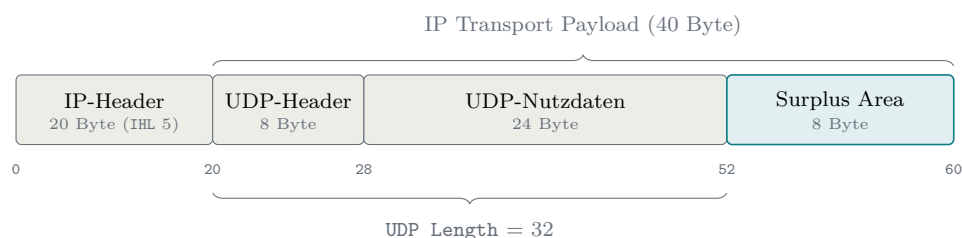


Abbildung 2.5: IP Transport Payload und Surplus Area eines Datagramms mit `Total Length` 60

Aus Abbildung 2.5 geht hervor, dass beide Größen unmittelbar hinter dem IP-

Header beginnen: **UDP Length** deckt von der IP Transport Payload nur UDP-Header und Nutzdaten ab, und die Surplus Area ist exakt der Rest bis zum Ende des IP-Datagramms. Im Beispiel umfasst die IP Transport Payload $60 - 20 = 40$ Byte, von denen **UDP Length** nur 32 beansprucht; die übrigen 8 Byte bilden die Surplus Area. Allgemein gilt

$$\text{Surplus-Länge} = \text{Total Length} - \text{Headerlänge} - \text{UDP Length}, \quad (2.3)$$

wobei die Obergrenze aus Gleichung 2.2 sicherstellt, dass diese Differenz nie negativ wird. Im Regelfall des heutigen Internets ist sie schlicht null: **UDP Length** und IP-Längenangaben enden am selben Byte, und es gibt keine Surplus Area. RFC 768 verlangt diese Gleichheit allerdings nicht; die Freiheit besteht seit seiner Veröffentlichung, RFC 9868 nutzt sie lediglich [2, Sec. 7].

Für den Beginn der Surplus Area schreibt RFC 9868 dabei bewusst keine Ausrichtung vor: Sie darf an jedem gültigen Byte-Offset anfangen, auch an einem ungeraden. Die Nutzdaten bleiben dadurch unangetastet, und das Ausrichtungsproblem wird, wie gleich zu sehen ist, innerhalb der Surplus Area gelöst. Im Ergebnis deutet RFC 9868 das Feld **UDP Length** in einen *trailer options offset* um: einen Zeiger auf das erste Byte hinter den Nutzdaten, an dem die Optionen beginnen [2, Sec. 7]. Genau darin liegt der inhaltliche Kern der Aktualisierung von RFC 768: Kein Feld ändert sein Format, ein Feld ändert seine Bedeutung [2].

Innerhalb dieses Bereichs legt RFC 9868 eine feste Struktur fest: Die Optionen nutzen stets die gesamte Surplus Area und beginnen mit der *Option Checksum* (OCS), einem 2 Byte großen Prüfsammenfeld an der ersten 2-Byte-Grenze der Area, gemessen am Beginn des IP-Datagramms. Beginnt die Surplus Area an einem ungeraden Offset, stellt genau ein Nullbyte vor dem OCS diese Ausrichtung her (das *Pad*). Hinter dem OCS folgen die eigentlichen Optionen im TLV-Format (Type-Length-Value); die Liste läuft bis zum Ende der Surplus Area oder endet vorzeitig mit EOL, worauf nur noch Nullbytes folgen [2, Sec. 8]. Diese Anatomie zeigt Abbildung 2.6 an einem Datagramm mit 5 Byte Nutzdaten und einer einzelnen Option:

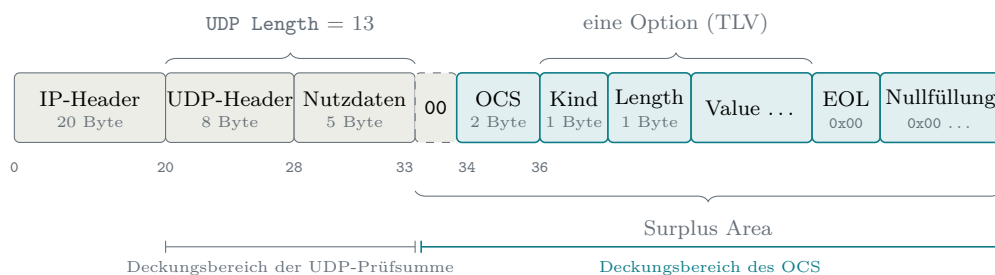


Abbildung 2.6: Anatomie eines Datagramms mit UDP Options

Aus Abbildung 2.6 geht hervor, wie die Bausteine ineinandergreifen: **UDP Length**

$= 8 + 5 = 13$ lässt die Surplus Area bei Byte $20 + 13 = 33$ beginnen. Dieser Offset ist ungerade, also rückt ein Pad-Byte das OCS auf die 2-Byte-Grenze bei Byte 34, und die erste Option beginnt bei Byte 36. Die unterste Ebene der Abbildung zeigt zugleich, dass sich die beiden Prüfbereiche lückenlos ergänzen: Die UDP-Prüfsumme endet an **UDP Length**, das OCS deckt genau den Teil des Datagramms, den die UDP-Prüfsumme nie sieht [2, Sec. 8].

Mit der Ausrichtung beginnt zugleich der normative Teil des RFC: Seine ersten verbindlichen Regeln überhaupt betreffen genau diese Stelle [2, Sec. 8]. Der Sender darf Optionen an jedem gültigen **UDP-Length**-Offset beginnen lassen; Ausrichtungsbytes vor dem OCS müssen null sein; und findet der Empfänger dort einen anderen Wert, muss er alle Optionen ignorieren und die Surplus Area stillschweigend verwerfen. Verworfen wird dabei nur die Surplus Area, nicht das Datagramm: Nutzdaten mit gültiger UDP-Prüfsumme werden weiterhin zugestellt [2, Sec. 8 und 9]. Die 2-Byte-Ausrichtung selbst begründet der RFC pragmatisch: Sie erspart der Prüfsummenberechnung das Vertauschen von Bytes [2, Sec. 8].

Ob ein Pad-Byte nötig ist, entscheidet allein die Parität des Startoffsets; es gibt nur zwei Fälle, kein Pad oder genau eines. Den Unterschied zeigt Abbildung 2.7 an zwei sonst gleichen Datagrammen, deren Nutzdaten sich um ein einziges Byte unterscheiden:

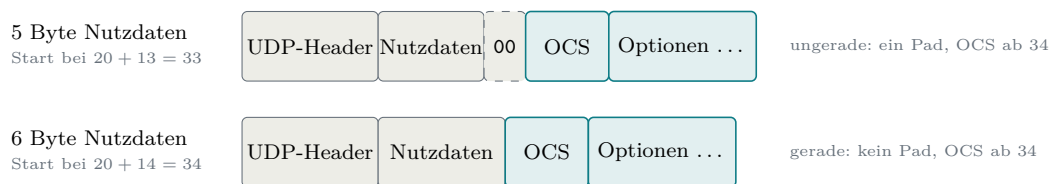


Abbildung 2.7: Paritätsentscheid für das Pad-Byte vor dem OCS

Aus Abbildung 2.7 geht hervor, dass bereits ein einziges Nutzdatenbyte über das Pad entscheidet: Bei einem IP-Header ohne Optionen liegt der Startoffset bei $20 + \text{UDP Length}$, sodass allein die Parität von **UDP Length** den Ausschlag gibt. Zwei Randfälle gehören zu diesem Bild. Fehlt die Surplus Area ganz, weil **UDP Length** die IP-Nutzlast vollständig ausfüllt, liegt schlicht ein gewöhnliches UDP-Datagramm vor: Es gibt nichts zu deuten, auch ein optionsfähiger Empfänger stellt die Nutzdaten wie ein Altempfänger zu, und die Optionsverarbeitung entfällt ersatzlos [2, Sec. 7]. Gebunden ist die Deutung als Optionen außerdem an eine Mindestgröße: Die Surplus Area trägt nur dann Optionen, wenn das ausgerichtete OCS vollständig hineinpasst. Ein Überschuss von einem einzelnen Byte (bei ungeradem Start: von zwei Byte) enthält daher schlicht keine Optionen und ist auch kein Fehlerfall [2, Sec. 8]. Nutzdaten braucht es dagegen keine: Auch ein Datagramm mit **UDP Length** 8, also leerem Nutzdatenteil, kann Optionen tragen [2, Sec. 8]; davon macht später die FRAG-Option Gebrauch, deren Fragmente ihre Daten vollständig in der Surplus

Area transportieren.

Das OCS selbst ist eine gewöhnliche Internet-Prüfsumme mit derselben Einerkomplementarithmetik, die Abschnitt 2.2.1 für die UDP-Prüfsumme beschreibt [14]. Zwei Eigenheiten unterscheiden die beiden. Erstens deckt das OCS genau den Bereich, den die UDP-Prüfsumme ausspart: die gesamte Surplus Area einschließlich des Pads, wobei das OCS-Feld für die Berechnung als null gilt [2, Sec. 9 und 10]. Zweitens geht zusätzlich die Länge der Surplus Area als vorzeichenloser 16-Bit-Summand in die Rechnung ein, obwohl dieser Wert an keiner Stelle des Pakets steht [2, Sec. 9]. Das folgt demselben Gedanken wie der Pseudo-Header aus Abschnitt 2.2.1: Wie die UDP-Prüfsumme dort die UDP `Length` einbezieht, bezieht das OCS die Surplus-Länge ein, und beide Seiten können diesen Summanden unabhängig voneinander aus den Längensfeldern ableiten. Der Sender trägt das invertierte Ergebnis in das Feld ein; der Empfänger summiert die Wörter der Surplus Area einschließlich des OCS-Felds sowie den Längensummanden und erwartet wie in Abschnitt 2.2.1 lauter Einsen, also `0xFFFF` [2, Sec. 9].

Diese Bauart hat einen praktischen Hintergrund: Manche fehlerhafte Middleboxes berechnen die UDP-Prüfsumme über die gesamte IP-Nutzlast statt nur bis UDP `Length`. Weil das OCS über die Surplus Area und deren Länge gebildet und anschließend invertiert wird, neutralisiert es den Beitrag der fälschlich einbezogenen Surplus Area: Mit gesetztem OCS fällt die UDP-Prüfsumme in beiden Rechnungen gleich aus, und solche Geräte stören sich nicht an dem Zusatzbereich [2, Sec. 9]. In erster Linie dient das OCS ohnehin dem Erkennen zufälliger Fehler und anderweitiger, nicht standardkonformer Nutzungen der Surplus Area; ein Schutz gegen gezielte Angriffe ist es ausdrücklich nicht [2, Sec. 9]. Wie die UDP-Prüfsumme ist das OCS zudem bedingt abschaltbar: Der Wert `0x0000` bedeutet ungenutzt, zulässig ist das nur, wenn auch die UDP-Prüfsumme null ist, und Implementierungen müssen standardmäßig ein echtes OCS setzen [2, Sec. 9]. Schlägt die Prüfung beim Empfänger fehl, gilt dieselbe Disposition wie beim Ausrichtungsfehler: Optionen ignorieren, Surplus Area still verwerfen, gültig geprüfte Nutzdaten dennoch zustellen [2, Sec. 9].

Die Optionen hinter dem OCS folgen schließlich dem TLV-Muster, dessen Syntax RFC 9868 bewusst an TCP anlehnt (Abschnitt 2.3): Ein 1 Byte großes Feld `Kind`, benannt nach dem TCP-Vorbild, identifiziert die Option; das 1 Byte große Feld `Length` zählt die Gesamtlänge der Option einschließlich `Kind` und `Length` selbst; dahinter steht der eigentliche Wert [2, Sec. 8 und 10]. Für die Größe des Werts ist das nur scheinbar eine Grenze: Im Standardformat fasst eine Option höchstens 254 Byte einschließlich `Kind` und `Length`; größere Optionen weichen auf ein erweitertes Format aus, in dem der Wert 255 im `Length`-Feld ein zusätzliches 16-Bit-Längensfeld ankündigt [2, Sec. 10]. Eine eigene Obergrenze für den Wert einer Option kennt RFC 9868 darüber hinaus bewusst nicht: Wie das Entwurfsprinzip der fehlenden

Längengrenze am Ende dieses Abschnitts festhält, begrenzt allein das UDP-Paket selbst den Optionsraum. Auch die beiden Ein-Byte-Codes aus Abschnitt 2.3 kehren als einzige Ausnahmen ohne Längensfeld wieder: **NOP** (Kind 1) dient mit impliziter Länge eins der Ausrichtung zwischen Optionen und darf sich dafür wiederholen, **EOL** (Kind 0) beendet die Liste vorzeitig; hinter **EOL** folgen bis zum Ende der Surplus Area nur noch Nullbytes [2, Sec. 8 und 10]. Für die Anzahl der Optionen gilt dasselbe Bild wie für ihre Länge: Eine feste Obergrenze gibt es nicht, die Liste endet erst am Ende der Surplus Area. Begrenzend wirken stattdessen die Wiederholungsregeln: Außer **FRAG**, **NOP** und den Experimentaloptionen soll jede Option höchstens einmal je Datagramm vorkommen, ein Empfänger wertet ohnehin nur die erste Instanz, und ein mehrfaches **FRAG** macht die gesamte Optionsliste ungültig [2, Sec. 10]. **NOP** wiederum soll höchstens siebenmal in Folge stehen, gerade genug für eine Ausrichtung an 8-Byte-Grenzen, und nicht als Ersatz für **EOL** samt Nullfüllung dienen [2, Sec. 11.2]. Variable Optionslängen und künftige Erweiterungen sind damit im Format selbst angelegt.

Ein bewusst kleines Rechenbeispiel führt beide Bausteine zusammen: Eine Surplus Area von 6 Byte beginne an gerader Grenze und enthalte hinter dem OCS zwei **NOP**, ein **EOL** und ein Nullbyte, als 16-Bit-Wörter gelesen also **0x0101** und **0x0000**. Der Sender addiert dazu den Längensummanden **0x0006**, erhält **0x0107** und trägt das invertierte Ergebnis **0xFEf8** als OCS ein. Der Empfänger summiert **0xFEf8**, **0x0101**, **0x0000** und **0x0006** zu **0xFFFF**: Die Prüfung geht auf, obwohl die Länge selbst nie übertragen wurde.

Da Empfänger, die UDP Options nicht unterstützen, ausschließlich die durch das UDP-Length-Feld begrenzten Nutzdaten verarbeiten, bleibt die Erweiterung für Endsysteme typischerweise abwärtskompatibel; Ausnahmen bei Bestandssystemen und das Verhalten von Zwischensystemen behandelt die Evaluation (Kapitel 6).

Wesentlich für die Einordnung ist, dass RFC 9868 kein neues Verhalten der Vermittlungsschicht einführt: Es definiert weder neue IP-Header-Felder noch verändert es die Verarbeitung von IPv4. Für die IP-Schicht sind die Bytes der *Surplus Area* reine IP Transport Payload innerhalb der durch **IP Total Length** begrenzten Datagramm-Größe; sie liegen lediglich hinter dem durch **UDP Length** markierten Ende der UDP-Nutzdaten. RFC 9868 ist damit ein reines Transport-Feature, das ausschließlich die zuvor ungenutzte Differenz zwischen **UDP Length** und der IP Transport Payload belegt [2].

Wie dieser Bereich genutzt werden darf, bindet RFC 9868 an sechs Entwurfsprinzipien, die ein gemeinsames Ziel haben: UDP soll durch die Optionen seinen Charakter nicht verlieren [2, Sec. 6]:

- **Zustandslos:** Nötiger Zustand gehört in die Anwendung oder in eine Bibliothek, die in ihrem Auftrag arbeitet. Die einzige, ausdrücklich begrenzte Ausnahme

ist die Reassemblierung von Fragmenten.

- **Unidirektional:** Die UDP-Schicht beantwortet keine Option selbstständig; Antworten löst die Anwendung aus. Andernfalls ließe sich die Schicht mit gefälschter Absenderadresse als Reflektor missbrauchen, denn UDP prüft die Herkunft eines Datagramms nicht. Verfahren, die Bidirektionalität benötigen, gehören in ein eigenes Dokument.
- **Keine eigene Längengrenze:** Für Optionen gilt allein die Grenze des UDP-Pakets. Feste Grenzen werden erfahrungsgemäß überschritten; der RFC verweist auf die 40 Byte des TCP-Optionsraums und die 32-Bit-Adressen von IPv4. Grenzen darf eine Implementierung setzen, nicht die Spezifikation.
- **Kein Protokollersatz:** Optionen liefern Funktionen für den eigenen Verkehr einer Anwendung; sie sollen NTP, ICMP (insbesondere Echo) und Netzwerk-Testgeräte weder ersetzen noch nachbauen.
- **Rahmenwerk statt Protokoll:** RFC 9868 definiert Optionsformate auch dann, wenn sie noch kein vollständiges Verfahren ergeben; die Nutzung regeln andere Dokumente. Für REQ/RES übernimmt das RFC 9869 [3], für TIME bleibt sie bewusst offen. Das entspricht dem Muster von TCP, dessen Basisnorm den Optionsmechanismus festlegt, während einzelne Optionen in eigenen Dokumenten folgten.
- **Voreinstellung wie beim Altempfänger:** Empfangene Pakete werden standardmäßig auch dann zugestellt, wenn Prüfsummen-, Authentifizierungs- oder Entschlüsselungsprüfungen der Optionen scheitern; einzige Ausnahme sind UDP-Fragmente. Strengeres Verhalten muss die Anwendung ausdrücklich anfordern. Optionen sind ein Angebot, kein Filter.

Der gemeinsame Nenner der sechs Prinzipien: RFC 9868 legt nur die Optionsformate fest. UDP bleibt ein minimales Transportprotokoll, und jede zusätzliche Fähigkeit wird von der Anwendung ausdrücklich aktiviert [2, Sec. 6].

An das letzte Prinzip knüpft die zentrale Zweiteilung der Optionen an, und sie folgt unmittelbar aus der Abwärtskompatibilität: Weil Altempfänger jede Surplus Area stumm überlesen, muss jede Option damit rechnen, ignoriert zu werden. *SAFE*-Optionen (Kind 0 bis 191) sind genau dafür entworfen: Sie lassen die Bedeutung der Nutzdaten unangetastet, ein Empfänger, der sie nicht versteht, ignoriert sie folgenlos; die bisher erwähnten Optionen einschließlich FRAG gehören in diese Klasse [2, Sec. 10 und 11]. *UNSAFE*-Optionen (Kind 192 bis 255) verändern dagegen die Nutzdaten selbst, etwa durch Kompression oder Verschlüsselung, und wären für einen Empfänger, der sie nicht versteht, unbrauchbar. RFC 9868 löst das mit

einer Transportbeschränkung: UNSAFE-Optionen dürfen nur in UDP-Fragmenten auftreten, deren Nutzdatenteil leer ist und deren Inhalt in der FRAG-Option der Surplus Area steckt. Ein Altempfänger sieht dann nur ein leeres Datagramm und verwirft die Daten still, statt sie falsch zu deuten; und auch ein optionsfähiger Empfänger, der eine UNSAFE-Option nicht unterstützt, muss die reassemblierten Nutzdaten still verwerfen [2, Sec. 10 und 12]. Die einzelnen Optionen und ihre Pflichten behandelt Kapitel 4.

Zwei Folgen dieser Prinzipien verdienen eine eigene Erwähnung. Erstens umspannt ein user datagram künftig gegebenenfalls mehrere IP-Datagramme, wenn es per FRAG-Option fragmentiert wird. Zweitens opfert der RFC an genau einer Stelle bewusst Kompatibilität: Ein Altempfänger sieht jedes UDP-Fragment als leeres Datagramm, ein optionsfähiger Empfänger das reassemblierte Datagramm mit Inhalt. Diese Zählungen gleicht der RFC nicht an, weil leere Datagramme als Lebenszeichen ohnehin unzuverlässig sind [2, Sec. 6].

2.5 Betriebssystem und Netzpfad

Ob ein Datagramm mit Surplus Area sein Ziel erreicht, entscheidet sich an zwei Stellen außerhalb des Protokolls: im Betriebssystem der beiden Endpunkte und auf dem Netzpfad dazwischen. Dieser Abschnitt stellt zunächst die zwei Socket-Sichten vor, mit denen ein Betriebssystem UDP-Verkehr an Anwendungen übergibt, und anschließend die Akteure des Pfades; am Ende fügt Abbildung 2.11 beides zu einem Gesamtbild zusammen. Allgemeine Netzwerkgrundlagen bleiben bewusst ausgespart: Es geht genau um die Kette, die die Messpfade dieser Arbeit tatsächlich durchlaufen.

Zwei Socket-Sichten. Der gewöhnliche Zugang zu UDP ist der *UDP-Socket*. Bei ihm übernimmt der Kernel die gesamte Protokollarbeit: Beim Empfang prüft er Länge und Prüfsumme, ordnet das Datagramm über den Zielport der richtigen Anwendung zu und übergibt ihr genau die Nutzdaten bis zur Grenze von `UDP Length` [4]. Eine Surplus Area bekommt die Anwendung nie zu sehen; ein Empfänger, der nur diese Sicht nutzt, ist damit genau der Altempfänger aus Abschnitt 2.4. Beim Senden gilt das Gegenstück: Der Kernel bemisst das IP-Datagramm stets exakt auf das UDP-Datagramm, hinter den Nutzdaten bleibt kein Platz. Ein UDP-Socket kann eine Surplus Area also weder erzeugen noch lesen. Wie dieser Normalweg im Kernel aussieht, zeigt Abbildung 2.8 mit den wichtigsten Funktionen je Schicht für beide Richtungen:

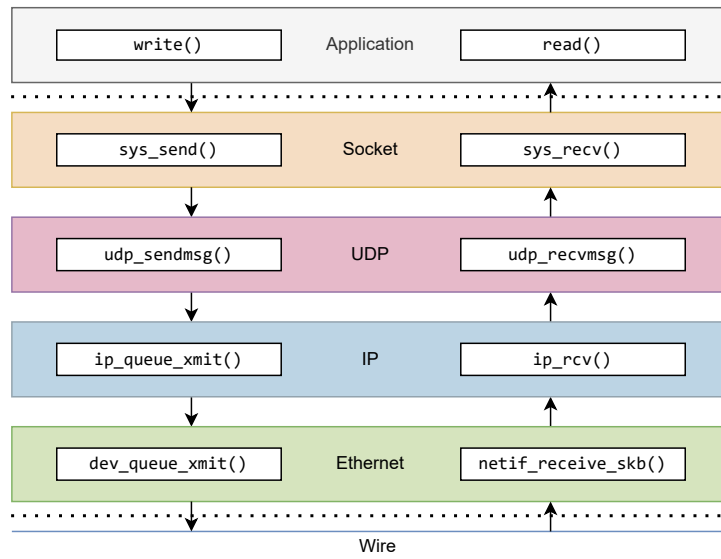


Abbildung 2.8: Sende- und Empfangsweg eines UDP-Datagramms durch den Linux-Kernel (aus [4])

Aus Abbildung 2.8 geht hervor, dass jede Schicht des Sendewegs ihr Gegenstück im Empfangsweg hat: `udp_sendmsg()` baut den UDP-Header und bemisst die Längen, `udp_rcvmsg()` liefert erst aus, nachdem der Kernel die UDP-Prüfsumme validiert hat [4]. Für diese Arbeit entscheidend ist, wo die zweite Sicht ansetzt: Der Raw-Sender speist sein fertig gebautes Datagramm eine Schicht tiefer ein, auf Höhe der IP-Schicht, und der Raw-Empfänger erhält seine Kopie, bevor der UDP-Pfad beginnt. Beide Abzweige zeichnen die folgenden Abbildungen nach.

Wer beides will, braucht die zweite Sicht: den *Raw Socket*. Er arbeitet unterhalb von UDP direkt mit IP-Datagrammen, und beim Senden erlaubt er mit der Socket-Option `IP_HDRINCL`, das IP-Datagramm mitsamt IP-Header selbst zu bauen [17]; erst damit lässt sich eine **Total Length** wählen, die über das Ende des UDP-Datagramms hinausreicht. Die vertauschten Rollen beider Sendewege zeigt Abbildung 2.9:

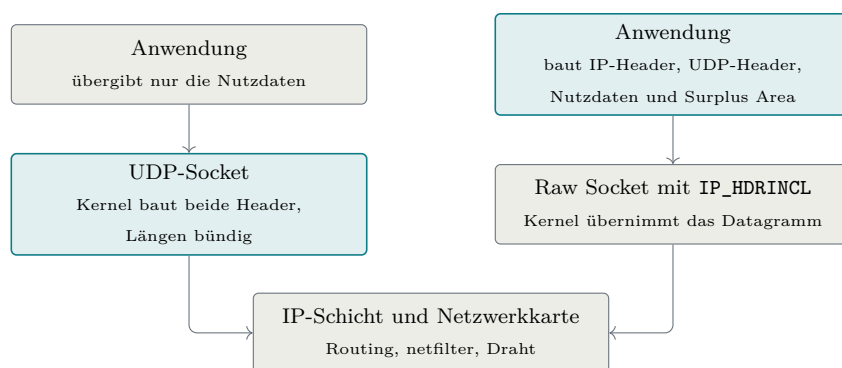


Abbildung 2.9: Die zwei Sendepfade durch den Linux-Kernel [4, 17]

Aus Abbildung 2.9 geht hervor, dass die Arbeitsteilung kippt: Am UDP-Socket

baut der Kernel beide Header und bemisst die Längen bündig, am Raw Socket liefert die Anwendung das fertige Datagramm ab, und nur auf diesem Weg entsteht hinter den Nutzdaten Platz für eine Surplus Area. Der Unterbau bleibt derselbe: Beide Wege münden in die IP-Schicht mit ihren netfilter-Stationen und enden an der Netzwerkkarte [4].

Ganz entlässt der Kernel den Raw-Sender allerdings nicht aus seiner Obhut, und diese Restarbeit prägt den Sendepfad der Implementierung. Routing, ARP und Link-Schicht bleiben Kernelsache, und im übergebenen IP-Header trägt der Kernel **Total Length** und die IP-Header-Prüfsumme stets selbst ein; UDP-Header und Surplus Area lässt er dagegen unangetastet [17]. Zugleich fragmentiert dieser Pfad nicht: Mit `IP_HDRINCL` ist ein Datagramm auf die MTU des Interfaces begrenzt, ein größerer Sendeaufruf schlägt fehl [17]. Für RFC 9868 ist das kein Verlust: IP-Fragmentierung sollen UDP-Anwendungen ohnehin vermeiden (Abschnitt 2.1) [8, Sec. 3.2], und große Nutzlasten zerlegt stattdessen die FRAG-Option auf Transportebene.

Beim Empfang entscheidet sich, an welcher Stelle ein Datagramm geprüft wird; den Weg vom Draht zu den beiden Sichten zeichnet Abbildung 2.10 nach:

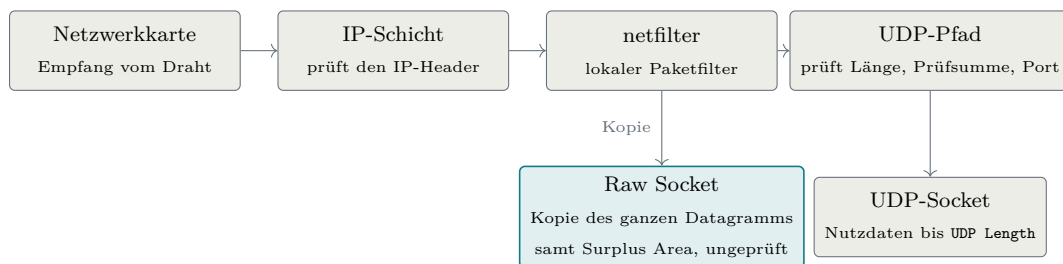


Abbildung 2.10: Der Empfangspfad durch den Linux-Kernel [4, 17]

Aus Abbildung 2.10 geht hervor, dass sich der Weg hinter dem Paketfilter gabelt: Der Raw Socket erhält eine Kopie des gesamten IP-Datagramms samt Surplus Area, und zwar bevor der UDP-Pfad des Kernels Länge, Prüfsumme und Portzuordnung geprüft hat [4, 17]. Ungeprüft heißt dabei vollständig ungeprüft: Auch ein Datagramm mit falscher UDP-Prüfsumme oder einer **UDP Length**, die über die IP-Nutzlast hinausweist, erreicht den Raw Socket unverändert. Die Prüfungen, die der Kernel dem UDP-Socket abnimmt, muss eine Raw-Empfangsschicht deshalb vollständig selbst ausführen, und zwar in der Reihenfolge, die RFC 9868 vorgibt: erst Mindestlänge und Längenkonsistenz des UDP-Headers, dann die UDP-Prüfsumme, erst danach Surplus-Ortung und OCS [2, Sec. 10 und 14]; wie die Implementierung das umsetzt, zeigt Kapitel 5. Zwei Arbeiten behält der Kernel dagegen auch hier. Die netfilter-Hooks bleiben beiden Sichten vorgeschaltet: Ein lokaler Paketfilter kann ein Datagramm verwerfen, bevor irgendeine Anwendung es sieht [4]. Und ankommende IP-Fragmente setzt die IP-Schicht bereits vor der Zustellung wieder zusammen, auch für Raw Sockets; eine Raw-Empfangsschicht muss die IPv4-Reassemblierung deshalb nicht

nachbauen [4, 17].

Akteure auf dem Netzpfad. Zwischen den Endpunkten begegnet das Datagramm einer zweiten Gruppe von Akteuren. Der Sammelbegriff dafür lautet *Middlebox*: ein Zwischensystem, das Pakete nicht nur weiterleitet, sondern liest, verändert oder verwirft [8, Sec. 3.5]. Für die Interpretation der späteren Messungen genügt der Sammelbegriff nicht, denn die Akteure setzen an verschiedenen Stellen an und hinterlassen verschiedene Spuren. Im Zugangsnetz ersetzt die NAT eines Heimrouters oder Hotspots Adressen und Ports und bindet jeden Fluss an einen Zustandseintrag mit Zeitschranke; ihre Provider-Variante *Carrier-Grade NAT* (CGNAT) teilt dieselben öffentlichen Adressen zusätzlich unter vielen Kunden auf. Firewalls verwerfen an Netzgrenzen und Endsystemen nach Regelwerk, etwa anhand von Ports und Protokollen. Leicht zu übersehen sind die letzten beiden Akteure: die Virtualisierungsschicht eines Testrechners, deren eigener Netz- und NAT-Stack Kopffelder auf erwartete Normwerte umschreiben kann, und das Transitnetz selbst, die AS-Kette zwischen den Zugangsnetzen, in der einzelne Netze filtern.

Keiner dieser Akteure kennt UDP Options: Was mit einer Surplus Area geschieht, ist stets Nebenwirkung von Regeln, die für gewöhnlichen UDP-Verkehr geschrieben wurden. Wie verschieden diese Nebenwirkungen ausfallen, zeigen drei Befunde, die Kapitel 6 belegt: Ein Mobilfunkzugang verwirft Datagramme mit Surplus Area klassenbasiert, ein VMware-NAT normalisiert die Kopffelder und entfernt die Surplus Area dabei, und der Transitpfad zu Hetzner transportiert sie unverändert. Zwei Eigenschaften prägen darüber hinaus die Messungen. Erstens arbeiten mehrere Akteure mit Zustand und Zeitschranken; derselbe Pfad kann sich je nach Vorgeschichte unterschiedlich verhalten [8, Sec. 3.5]. Zweitens lässt sich die Kette gezielt umgehen: Eine Kapselung, etwa in einen WireGuard-Tunnel, entzieht das innere Datagramm allen genannten Akteuren; Kapitel 6 nutzt genau das als Gegenprobe.

Diese Kette erklärt zugleich, warum RFC 9868 seine Optionen ausgerechnet in einem Bereich ablegt, den Altempfänger nie lesen. Weil Middleboxes bevorzugt durchlassen, was sie kennen, kann sich ein Protokoll praktisch nur dort weiterentwickeln, wo seine Neuerungen für sie unsichtbar bleiben; dieses Erstarren des Netzes unter seinen Zwischensystemen wird als *Ossifikation* bezeichnet. RFC 9868 rechnet ausdrücklich mit solchen Eingriffen: Fehlerhaft rechnende Middleboxes begründen Details der OCS-Berechnung (Abschnitt 2.4) [2, Sec. 9], und UDP-Relays können eine Surplus Area beim Weiterleiten vollständig abstreifen [2, Sec. 25.6].

Alle Stationen zusammen zeigt Abbildung 2.11: der Lebenslauf eines Datagramms von der sendenden Anwendung bis zu dem Raw-Empfänger, der die Surplus Area auswertet.

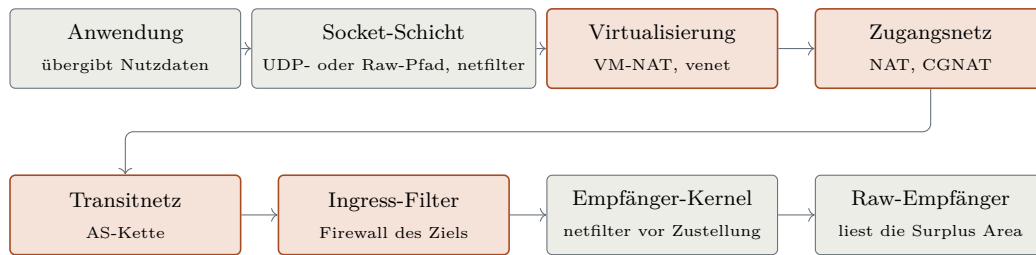


Abbildung 2.11: Der Lebenslauf eines Datagramms

Aus Abbildung 2.11 geht hervor, dass zwischen Socket-Schicht und Empfänger-Kernel vier Stationen liegen, die das Datagramm außerhalb der Endpunkte berühren: Sie greifen teils mit Zustand ein und sind von außen nicht direkt beobachtbar. Für die Feldmessungen folgt daraus die Interpretationsregel dieser Arbeit: Geht ein optionsbehaftetes Datagramm verloren oder kommt es verändert an, ist der Befund zuerst einer Station dieser Kette zuzuordnen, bevor er der Implementierung oder dem Standard angelastet wird; diese Zuordnung leisten in Kapitel 6 Messpunkte an beiden Enden des Pfades.

Bezogen auf diese Arbeit hat der Blick auf das Werkzeugumfeld eine letzte praktische Folge. Die verbreiteten Mitschnittwerkzeuge Wireshark und tshark kennen RFC 9868 in Version 4.6 nicht: Ein Mitschnitt zeigt eine Surplus Area kommentarlos als unbenannte Bytes hinter den Nutzdaten, ohne eigenes Feld und ohne Warnung. Für die Messkampagnen ist deshalb ein eigener Prüfer für aufgezeichnete Pakete unverzichtbar (Kapitel 6); zugleich zeigt der fehlende Dissektor, wie wenig verbreitet UDP Options im Ökosystem bislang sind.

Damit sind die Grundlagen beisammen: die beteiligten Protokolle, die Erweiterungsmuster, die Surplus Area mit ihren Entwurfsprinzipien sowie Betriebssystem und Netzpfad, auf denen jede Messung aufsetzt. Bevor die Arbeit mit ihnen Implementierung und Messungen bewertet, legt Kapitel 3 die Arbeitsweise offen, mit der diese Ergebnisse entstanden sind. Denn für alle folgenden Kapitel gilt derselbe Grundsatz: Eine Aussage eines Sprachmodells ist zunächst nur ein Kandidat; ob sie gilt, entscheiden RFC, Code und Messung.

3. Methodik und KI-gestützte Entwicklung

Große Sprachmodelle unterstützten die Auswertung des RFC-Primärtexts, den Entwurf und die Implementierung, die Analyse der Messkampagnen und die Prüfung von Zwischenergebnissen. Wissenschaftlich belastbar ist dieses Vorgehen nur, wenn die Arbeit offenlegt, wie Modellausgaben geprüft werden und wer die Ergebnisse verantwortet. Diese Regeln müssen vor der Ergebnisbewertung feststehen. Deshalb steht dieses Kapitel vor der RFC-Analyse.

Die KI-gestützte Entwicklung ist Teil der Methode, aber keine eigene Forschungsfrage. Dieses Kapitel beschreibt zunächst das Forschungsdesign und die Auswertung der Norm. Danach folgen die Rollen der Beteiligten, die bekannten Schwächen der Sprachmodelle und der Prüfprozess. Abschließend ordnen externe Referenzfälle das Vorgehen ein, und die Regeln für Reproduzierbarkeit und ihre Grenzen schließen das Kapitel ab. Die Darstellung beschreibt den aktuellen Sollprozess. Sie behauptet nicht, dass jede Stufe seit dem Beginn des Projekts in dieser Form bestand.

3.1 Forschungsdesign

Die Untersuchung verbindet die Auswertung der Norm, die Referenzimplementierung und eine empirische Pfaduntersuchung. Abbildung 3.1 zeigt, wie die beiden Forschungsfragen auf diesen Teilen aufbauen:

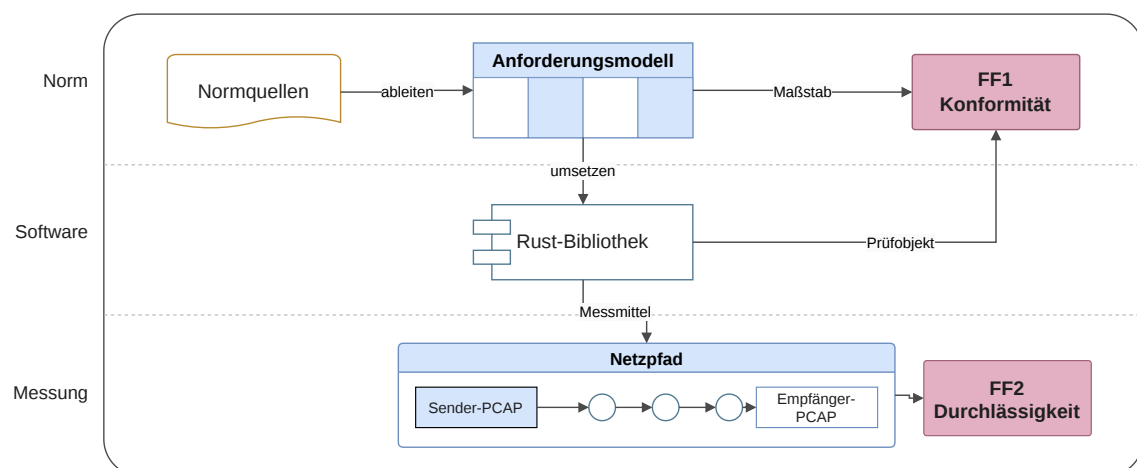


Abbildung 3.1: Aufbau der Untersuchung und Zuordnung der Forschungsfragen

Aus Abbildung 3.1 geht die Doppelrolle der Referenzimplementierung hervor: Für FF1 ist sie der Prüfgegenstand, und das aus den Normquellen abgeleitete Anforderungsmodell bildet den Maßstab; für FF2 dient dieselbe Implementierung als

Messmittel. Beide Bewertungen bleiben getrennt: Eine konforme Implementierung belegt keine Zustellbarkeit entlang realer Netzpfade, und eine unveränderte Zustellung belegt keine RFC-Konformität.

Die Forschungsfragen benötigen deshalb verschiedene Untersuchungseinheiten und Entscheidungsregeln. Tabelle 3.1 legt diese Regeln fest, bevor die Ergebnisse betrachtet werden:

Tabelle 3.1: Operationalisierung der Forschungsfragen

FF	Untersuchungs- einheit	Nachweis	Entscheidungsregel und Grenze
FF1	jede anwendbare normative Anforderung im Projektumfang	je nach Status: Normfundstelle sowie Quelltext und Prüfung oder Beleg einer technischen Grenze	Umsetzungsstand getrennt von vollständig, teilweise oder nicht erfüllbar; nicht anwendbar bleibt eine eigene Klasse
FF2	kontrolliertes Szenario; extern ein Paar aus Kontroll- und Optionsdatagramm je Pfad, Richtung und Messzeitpunkt	Paketmitschnitte an Sender und Empfänger, Sendeprotokoll und dokumentierte maschinelle Auswertung	Surplus erhalten, verändert oder entfernt, Paket verworfen oder Lauf ungültig; keine Übertragung auf unbeobachtete Pfade, Richtungen oder Zeitpunkte

Aus Tabelle 3.1 geht hervor, dass FF1 je Anforderung und FF2 je beobachtetem Paketpaar entschieden wird. Für FF1 wird zunächst die Anwendbarkeit bestimmt. Anwendbare **MUST**- und **SHOULD**-Aussagen gehen in die Bewertung ein; eine Abweichung von **SHOULD** benötigt eine Begründung. Eine **MAY**-Funktion wird nur bewertet, wenn sie für die Referenzimplementierung gewählt wurde. Nicht gewählte und außerhalb des Umfangs liegende Aussagen bleiben sichtbar, zählen aber weder als erfüllt noch als technisch unmöglich.

Der Matrixstatus und die FF1-Klasse beantworten verschiedene Fragen. „Implemented“ belegt den Umsetzungsstand. „Partially implemented“ bezeichnet eine Implementierungslücke und bestimmt die technische Erfüllbarkeit noch nicht. „Partial-in-userspace“ wird als teilweise erfüllbar und „Not-feasible-in-userspace“ als nicht erfüllbar gewertet. „Delegated“ gilt nur dann als vollständig erfüllbar, wenn der RFC die Pflicht der oberen Schicht zuweist und die Schnittstelle sie als dokumentierten Vertrag abbildet; andernfalls lautet die Klasse teilweise erfüllbar. Für implementierte Funktionen dienen Quelltext und Tests als Nachweis. Technische Grenzen werden stattdessen durch dokumentiertes Betriebssystem- oder Schnittstellenverhalten belegt.

Die kontrollierten lokalen Läufe werten jedes Szenario aus; die Szenarien sind über eigene Zielpoints markiert und können wie beim FRAG-Fall des RFC mehrere

Datagramme enthalten. Diese Läufe validieren die Messwerkzeuge und liefern Befunde zu den lokalen Topologien, beantworten aber nicht unmittelbar FF2 zu realen Netzpfaden. Dafür vergleichen die externen Kampagnen zeitnah gesendete, gleich große Kontroll- und Optionsdatagramme auf demselben gerichteten Pfad.

Ein externer Lauf ist nur auswertbar, wenn die Senderaufzeichnung den Versand und die erwartete Surplus Area belegt. Sender- und Empfängerlaufzeichnung werden paketweise verglichen. Die maschinelle Auswertung ordnet jedes Paar einer Ergebnis-kategorie zu: Die Surplus Area kommt erhalten, verändert oder entfernt an, oder das Paket wird verworfen; nicht eindeutig auswertbare Läufe, etwa bei abweichenden Paketzahlen, gelten als ungültig. IP-fragmentierte Datagramme gelten ebenfalls als ungültig, weil der Auswerter sie nicht zusammensetzt. Einzelne Vorversuche werden als solche gekennzeichnet und nicht zu allgemeinen Aussagen über das Internet oder ein nicht beobachtetes Gerät erweitert.

Sprachmodelle sind in beiden Untersuchungen Werkzeuge, aber keine bewertende Instanz. Maßgeblich sind die Normquellen, die Messdaten, ausführbare Prüfungen und die Freigabe durch den Autor. Der nächste Abschnitt beschreibt, wie aus den Normquellen ein prüfbares Anforderungsmodell entsteht.

3.2 RFC-Auswertung und schrittweise Umsetzung

Die normativen Protokollanforderungen stammen hauptsächlich aus RFC 9868. Ergänzende Anforderungen an Prüfsummen und das UDP-Format stammen aus RFC 1071 und RFC 768. Daneben enthält die Matrix ausdrücklich gekennzeichnete Projekt-, Schnittstellen- und Plattformvorgaben. Diese Trennung verhindert, dass eine lokale Entwurfsentscheidung nachträglich als Forderung des RFC erscheint.

Normative Schlüsselwörter wie **MUST**, **SHOULD** und **MAY** werden nach BCP 14¹ gelesen. Sie sind nur in Großschreibung normativ [18, 19]. RFC 9868 stellt Absätzen mit solchen Schlüsselwörtern zusätzlich das Zeichenpaar >> voran. Die eigene Zählung ergibt 103 markierte Absätze. Diese Zahl ist eine Kontrollzahl für mögliche Anforderungen, nicht die Zahl der abschließend anwendbaren Anforderungen.

Eine erste Zählung hatte nur 96 Absätze ergeben, weil tiefer eingerückte Markierungen übersehen wurden. Ein zweites Zählverfahren deckte den Fehler auf. Kontrollzahlen werden deshalb mit zwei getrennten Verfahren ermittelt. Jede gefundene Aussage erhält außerdem eine genaue Fundstelle, eine Normstärke, eine Beurteilung der Anwendbarkeit und eine Kennzeichnung des Projektumfangs.

RFC 9868 und RFC 9869 bleiben in der Quellenarbeit getrennt. RFC 9869 beschreibt die Pfad-MTU-Bestimmung (DPLPMTUD, *Datagram Packetization Layer*

¹ BCP 14 fasst RFC 2119 und RFC 8174 zusammen.

Path MTU Discovery) mit UDP-Optionen und liegt außerhalb des Implementierungsumfangs dieser Arbeit [3]. Zusätzlich wird der Errata-Stand des RFC Editors berücksichtigt [20]. Das verifizierte technische Erratum EID 8834 löst den Widerspruch zwischen den Abschnitten 10 und 14 zur Behandlung überlanger Optionen. EID 8708 ist als redaktionelles Erratum für eine spätere Dokumentausgabe zurückgestellt. Die Anforderungsmatrix verwendet diese beiden Einstufungen und dokumentiert die daraus abgeleitete Behandlung. Den Weg von den Quellen zur Matrix fasst Abbildung 3.2 zusammen:

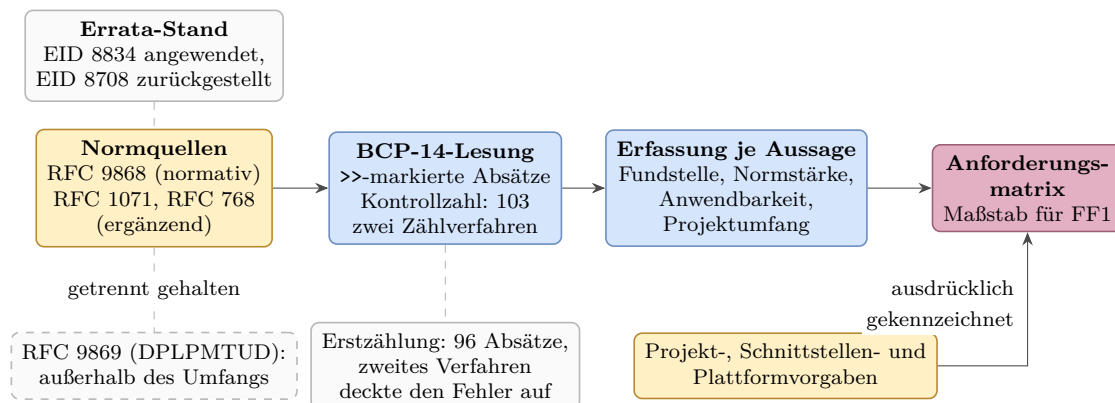


Abbildung 3.2: Von den Normquellen zur Anforderungsmatrix

Aus Abbildung 3.2 geht hervor, dass die Matrix aus zwei getrennt gehaltenen Zuflüssen entsteht: den normativen Quellen mit ihrem Errata-Stand und den ausdrücklich gekennzeichneten eigenen Vorgaben. Die Kontrollzahl sichert den normativen Zufluss mit zwei unabhängigen Zählverfahren ab, und RFC 9869 bleibt ohne Verbindung zur Matrix.

Die Umsetzung erfolgte schrittweise. Der Projektplan beginnt mit den Schritten 0 und 0.5 und reicht bis Schritt 18. Schritt 9 ging in Schritt 8 auf, Schritt 16 wurde mit der IPv6-Unterstützung aus dem Umfang entfernt. Die verbleibenden Schritte behandeln unter anderem Prüfsummen, Wire-Format, Parser, Serialisierung, typisierte Optionen, Raw Sockets, Fragmentierung, Schnittstellen und die Messumgebung. Die genaue Zuordnung und der jeweilige Abschlussnachweis stehen im Projektplan.² Diese Darstellung bildet den tatsächlichen Plan ab und ersetzt eine nachträglich vereinfachte Folge von neun Stufen. Welche Rollen bei der Erzeugung und Prüfung dieser Ergebnisse wirken, klärt der folgende Abschnitt.

3.3 Beteiligte und ihre Rollen

Der Autor entscheidet, gewichtet, gibt frei und verantwortet das Ergebnis. Claude Code von Anthropic dient vorwiegend als führendes Werkzeug für Entwürfe, Quelltext

² Implementierungs-Repository, Commit `f1cfe52`, `docs/plan/ROADMAP.md`.

und Analysen. Codex von OpenAI dient vorwiegend als getrennter Zweitprüfer mit dem Auftrag, Fehler und Gegenbeispiele zu suchen. Beide Programme verbinden ein Sprachmodell mit Dateizugriff, Kommandoausführung und dem Repository.

Eine Prüfung durch dasselbe Modell gilt nicht als getrennte Zweitprüfung. Auch die Prüfung durch ein Modell eines anderen Herstellers beweist keine Unabhängigkeit. Die Herstellertrennung ist eine bewusste Maßnahme gegen vollständig gleiche Fehlermuster, deren Wirkung in dieser Arbeit jedoch nicht gemessen wird. Änderungen durch den Zweitprüfer erfolgen nur in einem eigenen, vom Autor freigegebenen Arbeitsauftrag. Sie zählen dann nicht als Zweitprüfung desselben Ergebnisses.

Lean liefert maschinengeprüfte Beweise für handgeschriebene Modelle ausgewählter Invarianten des Wire-Formats, der OCS, der Fragmentierung und der Empfangslogik. Ein Prüfskript baut die Beweise und kontrolliert die verwendeten Axiome. Nicht bewiesen ist die Übereinstimmung des Rust-Codes mit diesen Modellen. Zulässig ist daher die Aussage, dass ausgewählte Invarianten in Lean-Modellen bewiesen sind. Die gesamte Implementierung ist nicht formal verifiziert. Dieses Zusammenspiel aus Erzeugung, Prüfung und Freigabe zeigt Abbildung 3.3:

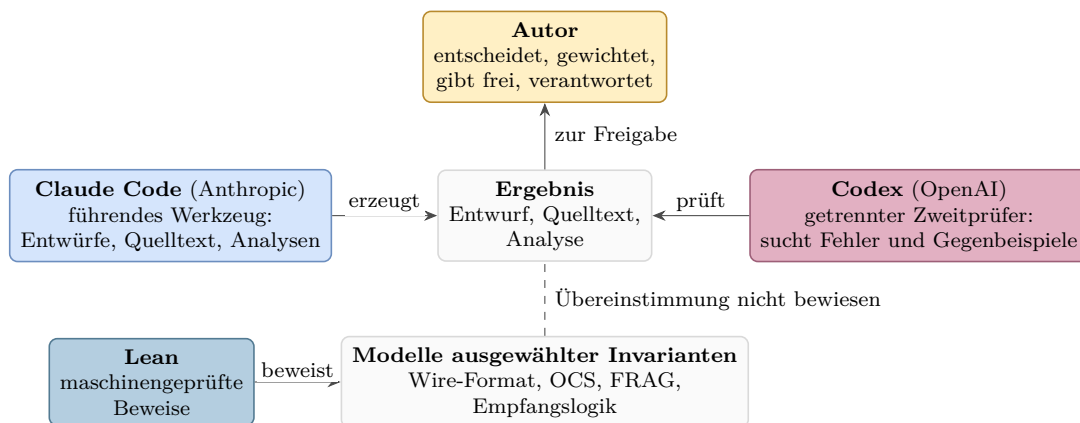


Abbildung 3.3: Rollen bei Erzeugung, Prüfung und Freigabe

Aus Abbildung 3.3 geht hervor, dass Erzeugung und Prüfung getrennt besetzt sind und jedes Ergebnis erst mit der Freigabe durch den Autor gilt. Lean wirkt dabei nur auf der Modellebene: Die gestrichelte Verbindung markiert, dass die Übereinstimmung des Rust-Codes mit den bewiesenen Modellen gerade nicht bewiesen ist.

Die Art der Prüfung richtet sich nach der Art der Aussage. Protokollaussagen werden gegen RFC-Primärtext und Errata geprüft. Aussagen über die Implementierung werden durch Quelltext und ausführbare Prüfungen gestützt. Pfadbefunde benötigen Paketmitschnitte und Kampagnenprotokolle. Literaturbehauptungen werden an der jeweiligen Primärquelle geprüft. Ein Hashwert belegt nur die Unverändertheit eines Artefakts, nicht die Wahrheit seines Inhalts.

Im Umgang mit den Modellen gilt die Haltung, sie wie fähige, aber unerfahrene Entwickler zu behandeln, deren Arbeit grundsätzlich geprüft wird. Aus diesem Grund

hat der Autor drei Arbeitsregeln schriftlich festgehalten:

1. Modellergebnisse werden gegen die jeweilige Primärquelle geprüft, nie gegen eigene Zusammenfassungen.
2. Aufträge werden neutral formuliert, ohne das Modell in eine gewünschte Richtung zu lenken.
3. Widerspruch wird ausdrücklich ermutigt: Ein Modell, das „nein“ sagt oder Unsicherheit meldet, ist wertvoller als eines, das gefällig zustimmt.

Diese Regeln beschreiben den Sollprozess. Welche Prüfungen tatsächlich stattfanden, wird nicht aus ihm abgeleitet, sondern aus den vorhandenen Protokollen rekonstruiert (Abschnitt 3.7). Warum diese Trennung nötig ist, zeigt der folgende Abschnitt.

3.4 Bekannte Schwächen der Sprachmodelle

Der Einsatz von Sprachmodellen bringt bekannte Schwächen mit, auf die die Methodik direkt antwortet.

Halluzination. Eine Halluzination ist ein überzeugend formulierter, aber falscher Inhalt. Kalai u. a. führen solche Fehler auf Trainings- und Bewertungsverfahren zurück, die Raten gegenüber eingestandener Unsicherheit belohnen [21].

Gefälligkeit. Sycophancy bezeichnet die Neigung, sich der erkennbaren Erwartung des Gegenübers anzuschließen. Sie wurde bei größeren Modellen und nach Training mit menschlicher Rückmeldung verstärkt beobachtet [22, 23]. Halluzination kann falsche Aussagen erzeugen. Gefälligkeit kann verhindern, dass ein Modell ihnen widerspricht.

Verfügbarkeit. Beide Sprachmodellwerkzeuge sind Cloud-Dienste. Sie können ausfallen, gedrosselt oder verändert werden. Die Ergebnisse liegen deshalb als Quelltext, Tests, Protokolle und Messdaten vor. Diese Artefakte bleiben ohne die Dienste prüfbar. Der genaue Entstehungsvorgang einer Modellausgabe ist wegen wechselnder Modelle und nichtdeterministischer Antworten dagegen nicht vollständig reproduzierbar. Abbildung 3.4 stellt jeder Schwäche ihre Wirkung und die methodische Antwort gegenüber:

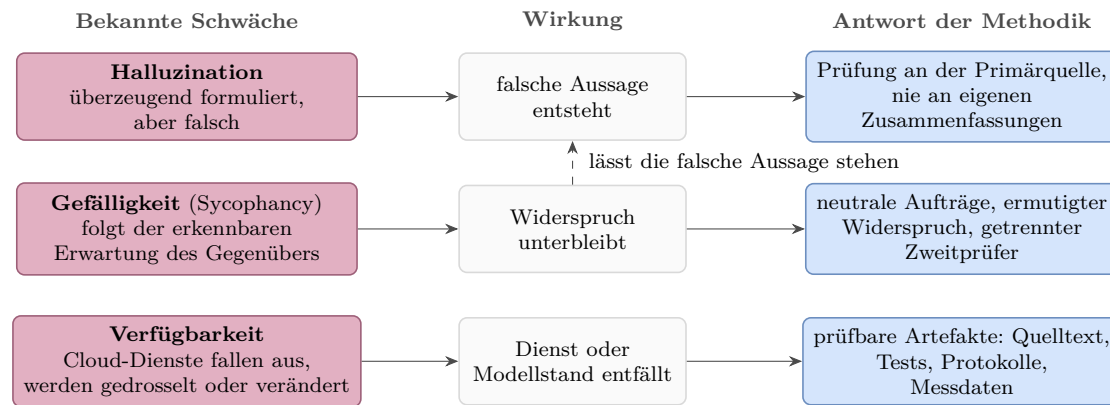


Abbildung 3.4: Schwächen der Sprachmodelle und methodische Antworten

Aus Abbildung 3.4 geht hervor, dass die ersten beiden Schwächen zusammenwirken: Halluzination erzeugt die falsche Aussage, und Gefälligkeit lässt sie unwidersprochen stehen. Die Antworten setzen deshalb nicht am Modell an, sondern an der Prüfinstanz und an der Form der Ergebnisse.

Die wichtigste Konsequenz ist einfach: Keine Aussage ist richtig, weil ein Modell sie behauptet. Ob die Implementierung die Norm erfüllt, wird mit der Konformitätsprüfung und der Testarchitektur beurteilt. Auch diese Prüfungen belegen nur die jeweils untersuchten Eigenschaften und keinen allgemeinen Korrektheitsbeweis.

Zwei Projektbefunde zeigen die praktische Bedeutung. Eine Gegenprüfung eigener Sekundärdokumente am RFC-Primärtext fand vier inhaltliche Abweichungen, obwohl die Implementierung an den betroffenen Stellen konform war. Eine Kampagnennotiz nannte außerdem 15 Kontrollmessungen, tatsächlich waren es 18. Drei Prüfinstanzen übersahen diesen Fehler zunächst. Eigene Zusammenfassungen bleiben daher Suchhilfen, und auch mehrere Prüfer sind keine Garantie.

3.5 Testarchitektur und Prüfprozess

Die aktuelle Prüfkette wird lokal gestartet, verbindet aber lokale und entfernte Prüfungen. Sie ist der Sollzustand vor dem Öffnen oder Aktualisieren eines Pull Requests. Historisch entstand diese Kette schrittweise. Aussagen über frühere Pull Requests beziehen sich deshalb nur auf die damals protokollierten Prüfungen.

Zu den deterministischen Prüfungen gehören Formatierung, statische Analyse, Dokumentationsbau, Unit-Tests und Integrationstests. Property-Tests erzeugen für jede Eigenschaft lokal 1024 Fälle. Neun zeitlich begrenzte Fuzz-Ziele suchen mit beliebigen Bytefolgen nach Abstürzen und verletzten Invarianten. Gefundene Gegenbeispiele werden als Regressionstests aufbewahrt. Die Lean-Prüfung baut die formalen Modelle und kontrolliert ihre Axiome.

Die systemnahen Stufen laufen auf dem entfernten Linux-Zielsystem. Dort prüfen

sie die Cross-Kompilierung, die Raw-Socket-Integration mit erhöhten Rechten und das tatsächlich gesendete Paketbild. Das lokal gestartete Skript und die Reihenfolge seiner Prüfschritte sind im Implementierungs-Repository festgehalten.³ Die kontinuierliche Integration (CI) wiederholt Formatierung, statische Analyse und Rust-Tests. Nach deren Erfolg baut sie die Lean-Modelle und stellt anschließend zwei getrennte Prüfanfragen: an Claude Code zu Codequalität und Sicherheit und an Codex zur RFC-9868-Semantik.

Eine besondere Gefahr sind Fehler, die Implementierung und Test gemeinsam haben. Sender und Empfänger der Bibliothek verwenden denselben Serialisierungscode. Ein beidseitig gleicher Fehler kann deshalb jeden Round-Trip-Test bestehen. Die Wire-Prüfung durchbricht diese Selbstbezüglichkeit: `tcpdump` zeichnet die tatsächlich gesendeten Bytes auf. Ein getrenntes Python-Programm dekodiert IPv4, UDP und die UDP-Optionen und berechnet die Prüfsummen mit eigener Faltung nach RFC 1071 [14] und einer eigenen CRC32C-Tabelle. `tshark` bestätigt zusätzlich die IPv4- und UDP-Felder. Es besitzt jedoch keinen Dissektor für UDP-Optionen.

Auch diese Prüfung ist nicht vollständig unabhängig. Die erwarteten Referenzbytes stammen aus der projekteigenen Formatdokumentation. Die Unabhängigkeit gilt für den Prüfcode, nicht für den Entwurf. Dasselbe gilt für die drei handgerechneten OCS-Referenzvektoren. Sie sind nicht palindromisch und wurden getrennt nachgerechnet, damit Bytevertauschungen auffallen. RFC 9868 enthält jedoch keinen veröffentlichten Referenzwert; eine externe Bestätigung dieser Vektoren steht noch aus.

Der aktuelle Sollprozess ist in Abbildung 3.5 als Sequenzdiagramm dargestellt: Jeder Beteiligte erhält eine senkrechte Lebenslinie, die schmalen Balken zeigen seine aktiven Phasen, durchgezogene Pfeile sind Aufträge und gestrichelte Pfeile Rückmeldungen. Auch diese Darstellung bleibt bewusst auf die Kontrollpunkte und die Grenze ihrer technischen Erzwingung reduziert:

³ Implementierungs-Repository, Commit `f1cfe52`, `scripts/pre-pr.sh`.

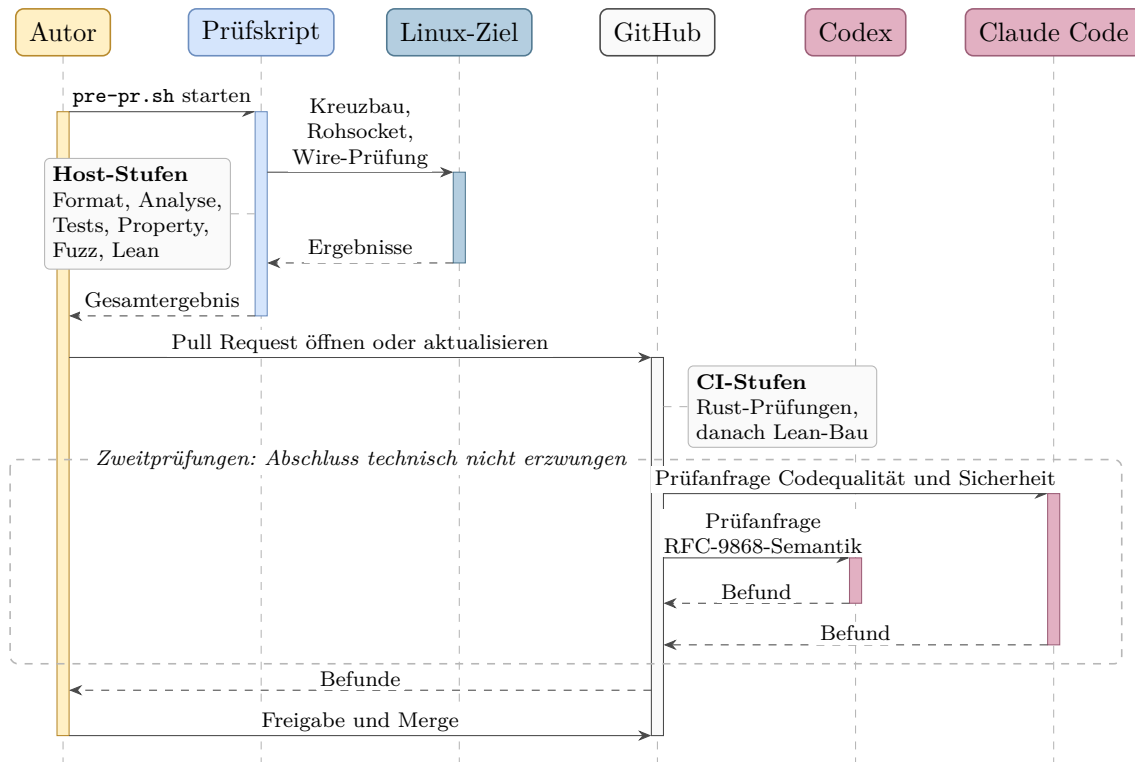


Abbildung 3.5: Prüfprozess im Sollzustand

Aus Abbildung 3.5 geht hervor, dass die Pfeilfolge nur die vorgesehene Reihenfolge vom Prüfskript über den Pull Request und die Rust/Lean-CI bis zur Freigabe durch den Autor zeigt; eine technische Merge-Bedingung ist sie nicht. Nach erfolgreicher Rust/Lean-CI werden getrennte Zweitprüfungen angefragt. Der gestrichelt umrahmte Bereich hebt hervor, dass auch deren Abschluss nicht erzwungen ist; das Thesis-Repository besitzt überhaupt keine solchen Prüfschritte. Pull Request 27 zeigt die Folge: Er wurde vor Abschluss aller Prüfungen zusammengeführt; ein Befund traf danach ein.⁴ Daher wird kein technisch erzwungenes Vier-Augen-Prinzip behauptet.

Diese Testarchitektur gehört zur Entwicklung. Die Pfaduntersuchung für FF2 verwendet die fertigen Programme, Sendeprotokolle sowie Paketmitschnitte an Sender und Empfänger. Ein bestandener Entwicklungstest sagt nichts über reale Netzpfade aus, und eine Pfadmessung ersetzt keinen Entwicklungstest. Das Projekt setzt außerdem kein Mutation-Testing am Rust-Quelltext ein. Gezielte Veränderungen an Paketmitschnitten prüfen nur, ob der jeweilige Auswerter bekannte Manipulationen erkennt. Der folgende Abschnitt ordnet diesen Prüfgedanken durch externe Vergleichsfälle ein.

⁴ Implementierungs-Repository, <https://github.com/ab7z/udp-transport-options/pull/27>.

3.6 Externe Referenzfälle

Das Muster dieser Methodik, dass ein generatives System vorschlägt, eine unabhängige Instanz prüft und ein Mensch freigibt, ist nicht projektspezifisch. Zur methodischen Einordnung dienen acht publizierte Fälle KI-gestützter Ergebnisse; sie bilden keine systematische Literaturübersicht und erlauben keine Aussage über die Häufigkeit oder Wirksamkeit eines bestimmten Prüfprozesses.

Vier Fälle zeigen die Bandbreite der Prüfinstanzen: AlphaFold wurde im blinden CASP14-Vergleich an experimentell bestimmten Proteinstrukturen gemessen [24]. Bei Davies u. a. lieferte das Modell Hypothesen und Kandidatenmuster, geprüft durch Gegenbeispielsuche und den klassischen Beweis der beteiligten Mathematiker [25]. Bei FunSearch bewertet ein deterministischer Auswerter jeden Programmkandidaten des Sprachmodells [26], und bei AlphaProof verifiziert der Lean-Kern jeden formalen Beweiskandidaten maschinell [27].

Vier weitere Fälle benennen ihre Prüfinstanzen besonders klar. Bei Knuths Zyklusproblem stammte die Konstruktion vom Modell, der Beweis vom Menschen und die Lean-Formalisierung von einem unabhängigen Dritten [28]. Die verbesserte Schranke zur Lage der Nullstellen der Riemannschen Zetafunktion wurde als Lean-Beweis formalisiert und intern wie extern begutachtet [29]. Beim Gegenbeispiel zur Einheitsabstands-Vermutung entnahmen Mathematiker die Konstruktion dem Modellprotokoll und führten den Nachweis selbst [30]. Der Physik-Preprint zur C-Parameter-Resummation legt im Abstract offen, dass Rechnungen, Numerik und Manuskript von Claude unter Aufsicht eines Physikers entstanden und gegen Monte-Carlo-Simulationen geprüft wurden [31]; dieser ausdrücklichen Offenlegung in der Publikation selbst folgt auch die vorliegende Arbeit (Abschnitt 1.6).

Als neunter Fall liegt die Migration der JavaScript-Laufzeit Bun von Zig nach Rust dem Softwareteil dieser Arbeit am nächsten. Das Projekt trennte Erzeugungs- und Prüfrollen, besetzte beide aber mit Modellen derselben Familie [32, 33]. Nach dem Merge wurde dennoch ein Fall undefinierten Verhaltens gemeldet; als Reaktion nahm das Projekt das Prüfwerkzeug Miri in die CI auf [34]. Daraus folgen zwei Lehren für diese Arbeit: Die Zweitprüfung besetzt bewusst ein Modell eines anderen Herstellers, und auch eine mehrstufige Prüfung gilt nicht als Garantie.

Alle neun Fälle teilen den Kern, dass Modellausgaben erst nach unabhängiger Prüfung und menschlicher Freigabe gelten; keiner stützt sein Ergebnis allein auf die Zahl der eingesetzten Agenten. Ein Erfolgsbeleg für die hier gewählte Modellkombination folgt daraus nicht. Das übertragbare Prinzip ist enger: Die Prüfinstanz muss zum Artefakt passen. Normtext, ausführbare Tests, formale Beweise, Messdaten und menschliche Freigabe erfüllen dabei unterschiedliche Aufgaben. Ob sie das im Projekt tatsächlich getan haben, muss dokumentiert sein; diese Dokumentation regelt der

folgende Abschnitt.

3.7 Reproduzierbarkeit und Grenzen

Aussagen über den Entstehungsprozess sind nur so belastbar wie ihre Protokollierung. Maßgeblich sind die vorhandenen Arbeitsprotokolle, also die Repository-Historie, die Prüf- und Kampagnenprotokolle sowie die Messdaten. Eine Prüfung, die dort nicht belegt ist, wird nicht nachträglich als erfolgt behauptet. Ein belegter Prüfschritt bedeutet umgekehrt nicht, dass die geprüfte Aussage allgemein wahr oder vollständig ist.

Eigene Zusammenfassungen und Arbeitsnotizen dienen nur als Suchhilfe. Die Prüfung erfolgt am Repository, am Primärtext oder an den Messdaten. Pannen werden nicht geglättet: Ein verlorener Prüfbericht wurde aus einem Sitzungsprotokoll wiederhergestellt und als Wiederherstellung gekennzeichnet. Eine versehentlich doppelte Zweitprüfung blieb ebenfalls erhalten. Auch Knuth berichtet, dass Zwischenstände einer Modellsitzung verloren gingen und die Fortschrittsdokumentation erneut eingefordert werden musste [28].

Technische Ergebnisse sind reproduzierbar, soweit Quellstand, Werkzeuge, Eingaben und Ausgaben archiviert sind. Die genaue Folge nichtdeterministischer Modellausgaben ist es nicht. Für frühe Projektphasen fehlen zudem vollständige Angaben zu Modellen und Konfigurationen. Diese Lücke bleibt bestehen und wird nicht rückwirkend durch den heutigen Sollprozess ersetzt.

Zum Abschluss dieses Kapitels bleiben die externe Bestätigung der OCS-Vektoren und die beschreibenden Kennzahlen der Zusammenarbeit offen. Die Wiedervorlage der Errata ist dagegen mit der Prüfung von EID 8834 und EID 8708 abgeschlossen. Kapitel 4 verwendet die hier festgelegten Quellen- und Bewertungsregeln, um aus RFC 9868 das Anforderungsmodell für FF1 abzuleiten.

4. RFC-Analyse und Anforderungsmodell

4.1 Anforderungen

4.1.1 Funktionale Anforderungen

4.1.2 Nicht-funktionale Anforderungen

4.1.3 Speichereffizienz und Zero-Copy

4.2 Existierende Implementierungen

4.2.1 Linux Kernel

4.2.2 Userspace-Implementierungen

4.3 Herausforderungen

4.3.1 Zugriff auf die Surplus Area

4.3.2 NAT-Traversal

4.3.3 Interoperabilität

4.3.4 Alternative Implementierungsansätze

4.4 Technologieauswahl

Die vorangegangene Analyse hat den Rahmen definiert, innerhalb dessen die Implementierung zu realisieren ist. Dieser Abschnitt überführt diese Analyse in die tragenden Technologie- und Methodenentscheidungen und begründet sie entlang der herausgearbeiteten Anforderungen.

4.4.1 Programmiersprache Rust

Die Wahl der Implementierungssprache wird nicht vorausgesetzt, sondern als Konsequenz der zuvor formulierten funktionalen und nicht-funktionalen Anforderungen entwickelt. Der Argumentationsbogen verläuft dabei in mehreren Schritten: von den Anforderungen, die das Vorhaben an eine geeignete Programmiersprache stellt, über die begriffliche Einordnung in die Klasse der Systemprogrammiersprachen und die Begründung der konkreten Sprachwahl bis zu einem Vergleich mit den Alternativen.

Aus dem RFC 9868 ergeben sich mehrere Anforderungen an die Programmiersprache, noch bevor eine konkrete Sprache benannt wird. Sie leiten sich unmittelbar aus den nicht-funktionalen Anforderungen ab. Zunächst wird ein direkter Zugriff auf *Raw Sockets* und damit auf rohe IP-Datagramme benötigt, da die Optionen von RFC 9868 in der *Surplus Area* jenseits der regulären UDP-Nutzdaten liegen und nur über selbst konstruierte Datagramme mit eigenem IP-Header (etwa mittels `IP_HDRINCL`) vollständig kontrolliert werden können. Hinzu kommt der Bedarf an einer bytengenauen Kontrolle in der *Surplus Area*, da die TLV-kodierten Optionen exakt an den durch den RFC vorgegebenen Bytegrenzen ausgerichtet sein müssen und jede implizite Umordnung oder Auffüllung durch die Sprache oder ihre Laufzeitumgebung die Korrektheit gefährden würde. Ebenso müssen potentiell bösartige eingehende Pakete ohne nennenswerten Laufzeit-Overhead verarbeitet werden, denn der Parser bildet die Vertrauensgrenze gegenüber dem Netz und darf weder durch präparierte Längfelder zum Absturz gebracht noch durch defensive Schutzmechanismen verlangsamt werden. Schließlich wird ein deterministisches Speicherverhalten ohne Garbage Collector vorausgesetzt, da solche Pausen das Laufzeitverhalten nicht-deterministisch machen würden. Diese Anforderungen bilden die Voraussetzungen zur Auswahl einer geeigneten Programmiersprache.

Sprachen, die diese Anforderungen erfüllen können, werden üblicherweise als *Systemprogrammiersprachen* zusammengefasst. Kennzeichnend für diese Klasse ist hardwarenahe Kontrolle über Speicher und Ausführung: direkter Zugriff auf Speicheradressen, Bitmuster und Betriebssystemschnittstellen ohne vermittelnde Abstraktionsschicht. Hinzu kommt eine explizite Speicherverwaltung ohne obligatorische

Laufzeitumgebung und insbesondere ohne Garbage Collector, sodass Allokation und Freigabe dem Programm und nicht einem im Hintergrund laufenden Prozess obliegen; der Quelltext wird zu nativem Maschinencode übersetzt, der ohne zwischengeschaltete virtuelle Maschine ausgeführt wird, woraus ein weitgehend deterministisches Laufzeitverhalten folgt. Eben diese Eigenschaften prädestinieren die Klasse für Aufgaben mit unmittelbarem Hardware- und Systembezug wie die Entwicklung von Betriebssystemen, Gerätetreibern und Netzwerkstacks. Genau in diese Schicht fällt die Implementierung eines Transportprotokoll-Mechanismus.

Die Definition wird von einer überschaubaren Gruppe etablierter Sprachen erfüllt, insbesondere von C, C++, Zig und Rust. Alle genannten Kandidaten bieten den geforderten hardwarenahen Zugriff und die Kompilation zu nativem Code; sie unterscheiden sich jedoch grundlegend darin, welche Sicherheitsgarantien sie dabei gewähren. Das entscheidende Kriterium ergibt sich daher nicht aus der Sprachklasse selbst, sondern aus der folgenden Anforderung: der sicheren Verarbeitung potentiell bösartiger Eingaben. Da der Parser die Vertrauensgrenze gegenüber dem Netz bildet, wird die standardmäßige Speichersicherheit zum entscheidenden Auswahlkriterium erhoben, denn eine Sprache, die Speicherfehler bereits konstruktiv ausschließt, verlagert eine ganze Klasse sicherheitskritischer Fehler von der Laufzeit in die Übersetzungsphase.

Dass diese Fehlerklasse kein akademisches Detail, sondern den weitaus größten Teil schwerwiegender Schwachstellen ausmacht, ist empirisch breit belegt: Sowohl bei Microsoft als auch im Chromium-Projekt werden übereinstimmend rund 70 Prozent der schwerwiegenden Sicherheitslücken auf Speichersicherheitsfehler zurückgeführt [35, 36], weshalb selbst die US-amerikanische National Security Agency (NSA) den Umstieg auf speichersichere Sprachen ausdrücklich empfiehlt und dabei Rust namentlich nennt [37]. Legt man die standardmäßige Speichersicherheit als Auswahlkriterium an das Feld an, so bleibt unter den etablierten Systemprogrammiersprachen Rust als einzige übrig, die diese Garantie ohne Garbage Collector und bereits zur Übersetzungsphase erbringt; die Wahl fällt folglich auf **Rust**.

Rust ist eine kompilierte, statisch und stark typisierte Systemprogrammiersprache, die ohne Garbage Collector und ohne obligatorische Laufzeitumgebung auskommt [38, 39]. Die statische Typisierung bedeutet dabei, dass die Typen aller Werte bereits zur Übersetzungszeit feststehen und vom Compiler geprüft werden, sodass eine ganze Klasse von Typfehlern vor der Ausführung und ohne Laufzeitkosten ausgeschlossen wird. Zugleich besteht über ein C-kompatibles *Foreign Function Interface* ein direkter Zugang zu den POSIX-Systemaufrufen, über die die benötigten *Raw Sockets* eingerichtet und die in der ersten Anforderung genannten IP-Datagramme kontrolliert

werden. Damit erfüllt Rust sämtliche Merkmale der zuvor entwickelten Definition: Es bietet hardwarenahe Kontrolle, verzichtet auf eine obligatorische Laufzeitumgebung und erzeugt deterministisch ausführbaren nativen Code.

Charakteristisch für Rust ist eine bewusste Auswahl der vorhandenen wie der absichtlich nicht vorhandenen Sprachmittel. Vorhanden sind das im nächsten Block ausführlich behandelte System aus *Ownership* und *Borrowing*; ferner algebraische Datentypen in Gestalt von `enum` mit vom Compiler auf Vollständigkeit geprüfem *Pattern Matching*; eine auf dem Typ `Result` aufbauende explizite Fehlerbehandlung; eine starke statische Typisierung; sowie das Schlüsselwort `unsafe` als bewusst markierte und eng begrenzte Brücke für jene Operationen, deren Sicherheit der Compiler nicht selbst nachweisen kann. Nicht vorhanden sind hingegen ein `null`-Wert, implizite Typkonvertierungen, klassische *Exceptions* sowie die Implementierungsvererbung objektorientierter Sprachen.

Gerade das Fehlen dieser Konstrukte verringert die Zahl der Fehlerquellen in einem sicherheitskritischen Parser erheblich. Diese Sprachmittel prägen den Aufbau der vorliegenden Implementierung unmittelbar. Sämtliche `unsafe`-Blöcke sind in der Socket-Schicht (`src/socket/`) konzentriert und dort hinter sichere Wrapper gekapselt, sodass der übrige Code ausschließlich mit sicheren Abstraktionen arbeitet.

Algebraische Datentypen und *Pattern Matching* bilden die TLV-kodierten Optionen der *Surplus Area* typsicher und erschöpfend ab, und die `Result`-basierte Fehlerbehandlung zwingt dazu, fehlerhafte oder bösartige Eingaben an jeder Verarbeitungsstelle explizit zu behandeln. Parser, Serializer und Prüfsummenberechnung sind dabei bewusst handgeschrieben und greifen nicht auf etablierte Bibliotheken wie `nom` oder `zerocopy` zurück, da die Mechanik von RFC 9868 selbst Gegenstand der Untersuchung ist und durch eine fertige Abstraktion nicht verdeckt werden soll.

Den Kern der Sprachwahl bildet die Art und Weise, in der Rust *Memory Safety* erzwingt: zum überwiegenden Teil bereits zur Übersetzungszeit durch das Typsystem, ergänzt um wenige, eng umrissene Laufzeitprüfungen. Unter *Memory Safety* wird die Abwesenheit einer Klasse von Speicherzugriffsfehlern verstanden, zu der insbesondere der *Buffer Overflow* (ein Zugriff jenseits der Grenzen eines reservierten Speicherbereichs), das *Use-after-free* (ein Zugriff auf bereits freigegebenen Speicher) und der *Data Race* (ein unsynchronisierter, nebenläufiger Zugriff mehrerer Ausführungsstränge, von denen mindestens einer schreibt) zählen. Rust begegnet diesen Fehlerklassen auf zwei Wegen: Das *Use-after-free* und der *Data Race* werden durch ein statisches Regelwerk ausgeschlossen, das aus drei aufeinander aufbauenden Konzepten besteht und das der Compiler vor der Codeerzeugung durchsetzt [39, 40]; der Zugriff jenseits der Grenzen eines Speicherbereichs hingegen wird, soweit er nicht ohnehin statisch

ausgeschlossen ist, durch automatisch eingefügte Bereichsprüfungen abgefangen, die bei einer Verletzung einen kontrollierten Programmabbruch (*panic*) auslösen, statt wie in C undefiniertes Verhalten zuzulassen [39, Kap. 9].

Das erste ist *Ownership*: Jeder Wert besitzt zu jedem Zeitpunkt genau einen Eigentümer; wird der Wert einer anderen Bindung zugewiesen oder an eine Funktion übergeben, so geht das Eigentum über (*move*), und die ursprüngliche Bindung ist anschließend nicht mehr verwendbar, was ein doppeltes Freigeben desselben Speichers ausschließt; verlässt der Eigentümer seinen Gültigkeitsbereich, wird der zugehörige Speicher deterministisch freigegeben, ohne dass es dafür eines Garbage Collectors bedürfte.

Das zweite ist *Borrowing*: Anstelle einer Eigentumsübertragung kann ein Wert über Referenzen geborgt werden, wobei zu jedem Zeitpunkt entweder beliebig viele geteilte, ausschließlich lesende Referenzen oder genau eine exklusive, schreibende Referenz bestehen dürfen, niemals jedoch beides zugleich; diese Regel schließt *Data Races* konstruktiv aus, da ein unsynchronisierter schreibender Zugriff bei gleichzeitig bestehenden weiteren Referenzen nicht typisierbar ist.

Das dritte sind *Lifetimes*: Sie binden die Gültigkeitsdauer jeder Referenz an jene des referenzierten Wertes und stellen so sicher, dass keine Referenz ihren Referenten überdauert, womit *Use-after-free* ausgeschlossen wird.

Die Gesamtheit dieser Regeln erzwingt der *Borrow Checker* vollständig zur Übersetzungszeit: Code, der gegen eine der Regeln verstößt, wird nicht übersetzt, und es entstehen keinerlei Laufzeitkosten für diese Prüfung. Das nachstehende Beispiel zeigt den Mechanismus an einem minimalen Fall, in dem eine geteilte und eine exklusive Referenz koexistieren sollen.

```
1 let mut buf = vec![1u8, 2, 3];
2 let view = &buf[0];           // geteilte (lesende) Referenz auf buf
3 buf.push(4);                  // Fehler: exklusiver Borrow, solange '
    view' lebt
4 println!("{view}");           // Nutzung von 'view' haelt den Borrow
    am Leben
```

Der Borrow Checker verweigert hier die Übersetzung mit dem Fehlercode E0502, weil `buf.push` eine exklusive Referenz auf den Puffer benötigt, während die geteilte Referenz `view` noch gebraucht wird; ein in C oder C++ an dieser Stelle möglicher Zugriff über einen durch die Reallokation ungültig gewordenen Zeiger ist damit bereits zur Übersetzungszeit ausgeschlossen:

```
1 --> src/main.rs:4:1
```

```

2 |
3 | let view = &buf[0];          // geteilte (lesende) Referenz auf
  | buf
4 |          --- immutable borrow occurs here
5 | buf.push(4);                // Fehler: exklusiver Borrow,
  | solange 'view' lebt
6 | ~~~~~ mutable borrow occurs here
7 | println!("{view}");         // Nutzung von 'view' haelt den
  | Borrow am Leben
8 |          ---- immutable borrow later used here
9 |
10 | For more information about this error, try 'rustc --explain E0502'.

```

Darauf aufbauend lässt sich die Wahl gegen die übrigen Kandidaten der Klasse präzise begründen, ohne deren jeweilige Stärken zu übergehen.

C bietet maximale, unverstellte Kontrolle über Speicher und Hardware und besitzt das mit Abstand reife Ökosystem für hardwarenahe Netzprogrammierung; diese Kontrolle wird jedoch ohne jede sprachseitige Sicherheitsgarantie gewährt, denn weder Schranken- noch Lebensdauerprüfungen für Zeiger sind Bestandteil der Sprachdefinition, und der Standard kennt eine große Zahl von Fällen mit *Undefined Behavior*, darunter gerade Pufferüberläufe und Zugriffe auf freigegebenen Speicher [41].

C++ erweitert dieses Fundament um mächtige Abstraktionsmittel und mildert mit *RAII* sowie Smart Pointern einen Teil der manuellen Speicherverwaltung, bleibt jedoch unsicher per Voreinstellung, da diese Werkzeuge optional sind und neben den unsicheren C-Altlasten fortbestehen, und erkaufte seine Ausdrucksstärke mit erheblicher Sprachkomplexität.

Zig ist eine moderne, bewusst einfach und explizit gehaltene Systemprogrammiersprache, die mit *comptime* eine ausdrucksstarke Auswertung zur Übersetzungszeit bietet und versteckte Allokationen vermeidet, indem Speicher ausschließlich über explizit übergebene Allocator-Objekte angefordert wird [42]; eine compilergarantierte Speichersicherheit nach Art des Borrow Checkers kennt Zig jedoch nicht, sondern erkennt nur eine Teilmenge unzulässiger Zugriffe durch Laufzeitprüfungen in den sicheren Build-Modi, die in den auf Geschwindigkeit oder Größe optimierten Modi entfallen, und hat mit Version 0.16.0 noch keine stabile Fassung erreicht [42]¹.

¹ Welche praktische Relevanz dieser Unterschied besitzt, zeigt die JavaScript-Laufzeit Bun, lange das größte Produktivprojekt in Zig: Nachdem allein in Version 1.3.14 vierzig Speicherlecks und zahlreiche Abstürze einzeln behoben worden waren, portierte das Team um Jarred Sumner den Kern von Zig nach Rust, um derartige Fehler strukturell auszuschließen, statt sie dauerhaft nachzubessern [32, 43]. Bun ist dabei kein Einzelfall: Auch der Paketmanager pnpm portiert seine Installations-Engine offiziell nach Rust; das zunächst separate Projekt paquet wird seit Juli 2026 im pnpm-Hauptrepositorium weitergeführt [44].

Rust nimmt in diesem Feld die Stellung ein, C-vergleichbare Kontrolle und Performance mit einer bereits zur Übersetzungszeit erzwungenen Speicher- und Data-Race-Freiheit zu verbinden; der Preis dafür ist die Einarbeitung in das Ownership-Modell, die jedoch durch präzise Compiler-Diagnosen unterstützt wird.

Abschließend stellt Tabelle 4.1 die entscheidungsrelevanten Eigenschaften der vier Kandidaten gegenüber.

Tabelle 4.1: Vergleich der betrachteten Systemprogrammiersprachen anhand der für die RFC 9868-Implementierung maßgeblichen Kriterien

Kriterium	C	C++	Zig	Rust
Speichersicherheit standardmäßig	nein	nein	nur in sicheren Build-Modi	✓ (Compiler)
Data-Race-Freiheit zur Übersetzungszeit	nein	nein	nein	✓
Speicherverwaltung	manuell	manuell, RAII	manuell, explizite Allocator	Ownership, kein GC
Laufzeit-Overhead	keiner, da ungeprüft	keiner, da ungeprüft	nur in sicheren Build-Modi	minimal (Be-reichsprüfung)
Compiler-Diagnostik als Rückkopplungs-schleife	begrenzt	begrenzt, oft unübersicht-lich	solide	ausgeprägt, mit stabilen Fehlercodes
Reife des Ökosystems (hardwarenahe Netze)	sehr hoch	sehr hoch	gering, noch nicht stabil	hoch, wachsend

Über die reinen Sicherheitsgarantien hinaus erweist sich der Rust-Compiler als eine starke, deterministische und maschinenlesbare Rückkopplungsschleife, was im Rahmen dieser Arbeit aus einem zweiten Grund bedeutsam ist. Seine Diagnosen benennen den Fehlerort nicht nur präzise, sondern tragen stabile Fehlercodes der Form `E0xxx` und ergänzen häufig konkrete, mit `help:` eingeleitete Korrekturvorschläge; ergänzt wird dies durch `cargo check` für eine schnelle Prüfung ohne vollständige Codegenerierung und durch den Linter `clippy` für eine weitergehende statische Analyse stilistischer und semantischer Schwachstellen [40].

Diese Eigenschaft gewinnt über den reinen Komfort beim Entwickeln hinaus an Bedeutung: Ein Compiler, der Fehler nicht nur meldet, sondern präzise lokalisiert und benennt, stellt ein hartes, automatisch und ohne menschliches Zutun prüfbares

Ziel bereit, gegen das sich jede einzelne Codeänderung verifizieren lässt, sodass fehlerhafte Konstruktionen nicht erst zur Laufzeit, sondern bereits bei der Übersetzung zurückgewiesen werden; genau das ist der Mechanismus, den maschinell erzeugter Quelltext benötigt. Wie sich diese Verbindung aus statischen Garantien und maschinenlesbarem Feedback methodisch nutzen lässt und in welchen Entwicklungsrahmen sie sich einfügt, behandelt Kapitel 3.

5. Entwurf und Implementierung

5.1 Architektur

5.1.1 Architekturüberblick

5.1.2 Projektstruktur

5.1.3 Datenmodell der UDP Options

5.2 TLV-Parser und Serializer

5.2.1 Parsing-Algorithmus

5.2.2 Serialisierung

5.3 Option Checksum (OCS)

5.3.1 Berechnung

5.3.2 Validierung

5.4 Raw-Socket-Zugriff und Plattform-Spezifika

5.4.1 Socket-Erstellung und Paketempfang

5.4.2 Plattform-Spezifika unter Linux

5.4.3 Build-Konfiguration

6. Evaluation und Diskussion

6.1 Test- und Messkonzept

6.2 Beobachtbarkeit und Middlebox- Verträglichkeit auf realen Pfaden

6.3 Soll-Ist-Abgleich mit RFC 9868

7. Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

7.2 Ausblick

7.3 Schlusswort

Literaturverzeichnis

- [1] J. Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force, Aug. 1980. DOI: [10.17487/RFC0768](https://doi.org/10.17487/RFC0768). URL: <https://www.rfc-editor.org/info/rfc768>.
- [2] J. Touch. *Transport Options for UDP*. RFC 9868. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9868](https://doi.org/10.17487/RFC9868). URL: <https://www.rfc-editor.org/info/rfc9868>.
- [3] G. Fairhurst und T. Jones. *Datagram Packetization Layer Path MTU Discovery (DPLPMTUD) for UDP Options*. RFC 9869. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9869](https://doi.org/10.17487/RFC9869). URL: <https://www.rfc-editor.org/info/rfc9869>.
- [4] A. Stephan und L. Wüstrich. „The Path of a Packet Through the Linux Kernel“. In: *Seminar Innovative Internet Technologies and Mobile Communications (IITM), WS 23*. Technische Universität München, Chair of Network Architectures und Services, 2024, S. 89–93. DOI: [10.2313/NET-2024-04-1_16](https://doi.org/10.2313/NET-2024-04-1_16). URL: https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2024-04-1/NET-2024-04-1_16.pdf (besucht am 13.08.2026).
- [5] J. Postel. *Internet Protocol*. RFC 791. Internet Engineering Task Force, Sep. 1981. DOI: [10.17487/RFC0791](https://doi.org/10.17487/RFC0791). URL: <https://www.rfc-editor.org/info/rfc791>.
- [6] S. Deering und R. Hinden. *Internet Protocol, Version 6 (IPv6) Specification*. RFC 8200. Internet Engineering Task Force, Juli 2017. DOI: [10.17487/RFC8200](https://doi.org/10.17487/RFC8200). URL: <https://www.rfc-editor.org/info/rfc8200>.
- [7] F. Gont, R. Atkinson und C. Pignataro. *Recommendations on Filtering of IPv4 Packets Containing IPv4 Options*. RFC 7126. Internet Engineering Task Force, Feb. 2014. DOI: [10.17487/RFC7126](https://doi.org/10.17487/RFC7126). URL: <https://www.rfc-editor.org/info/rfc7126>.
- [8] L. Eggert, G. Fairhurst und G. Shepherd. *UDP Usage Guidelines*. RFC 8085. Internet Engineering Task Force, März 2017. DOI: [10.17487/RFC8085](https://doi.org/10.17487/RFC8085). URL: <https://www.rfc-editor.org/info/rfc8085>.
- [9] W. Eddy. *Transmission Control Protocol (TCP)*. RFC 9293. Internet Engineering Task Force, Aug. 2022. DOI: [10.17487/RFC9293](https://doi.org/10.17487/RFC9293). URL: <https://www.rfc-editor.org/info/rfc9293>.

- [10] V. Cerf, Y. Dalal und C. Sunshine. *Specification of Internet Transmission Control Program*. RFC 675. Internet Engineering Task Force, Dez. 1974. DOI: [10.17487/RFC0675](https://doi.org/10.17487/RFC0675). URL: <https://www.rfc-editor.org/info/rfc675>.
- [11] J. Iyengar und M. Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000. Internet Engineering Task Force, Mai 2021. DOI: [10.17487/RFC9000](https://doi.org/10.17487/RFC9000). URL: <https://www.rfc-editor.org/info/rfc9000>.
- [12] M. Bishop. *HTTP/3*. RFC 9114. Internet Engineering Task Force, Juni 2022. DOI: [10.17487/RFC9114](https://doi.org/10.17487/RFC9114). URL: <https://www.rfc-editor.org/info/rfc9114>.
- [13] J. F. Kurose und K. W. Ross. *Computer Networking: A Top-Down Approach*. 8. Aufl. Pearson, 2021.
- [14] R. Braden, D. Borman und C. Partridge. *Computing the Internet Checksum*. RFC 1071. Internet Engineering Task Force, Sep. 1988. DOI: [10.17487/RFC1071](https://doi.org/10.17487/RFC1071). URL: <https://www.rfc-editor.org/info/rfc1071>.
- [15] C. Huitema. *Teredo: Tunneling IPv6 over UDP through Network Address Translations (NATs)*. RFC 4380. Internet Engineering Task Force, Feb. 2006. DOI: [10.17487/RFC4380](https://doi.org/10.17487/RFC4380). URL: <https://www.rfc-editor.org/info/rfc4380>.
- [16] D. Thaler. *Teredo Extensions*. RFC 6081. Internet Engineering Task Force, Jan. 2011. DOI: [10.17487/RFC6081](https://doi.org/10.17487/RFC6081). URL: <https://www.rfc-editor.org/info/rfc6081>.
- [17] Linux man-pages project. *raw(7): IPv4 raw sockets*. Linux man-pages 6.18. URL: <https://man7.org/linux/man-pages/man7/raw.7.html> (besucht am 17.08.2026).
- [18] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Internet Engineering Task Force, März 1997. DOI: [10.17487/RFC2119](https://doi.org/10.17487/RFC2119). URL: <https://www.rfc-editor.org/info/rfc2119>.
- [19] B. Leiba. *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174. Internet Engineering Task Force, Mai 2017. DOI: [10.17487/RFC8174](https://doi.org/10.17487/RFC8174). URL: <https://www.rfc-editor.org/info/rfc8174>.
- [20] RFC Editor. *RFC Errata for RFC 9868*. RFC Editor. 2026. URL: <https://www.rfc-editor.org/errata/rfc9868> (besucht am 13.08.2026).
- [21] A. T. Kalai, O. Nachum, S. S. Vempala und E. Zhang. *Why Language Models Hallucinate*. arXiv:2509.04664. 2025. URL: <https://arxiv.org/abs/2509.04664> (besucht am 13.08.2026).

- [22] E. Perez u. a. *Discovering Language Model Behaviors with Model-Written Evaluations*. arXiv:2212.09251. 2022. URL: <https://arxiv.org/abs/2212.09251> (besucht am 13.08.2026).
- [23] M. Cheng, C. Lee, P. Khadpe u. a. „Sycophantic AI decreases prosocial intentions and promotes dependence“. In: *Science* 391.6792 (2026). Artikelkennung eaec8352. DOI: [10.1126/science.aec8352](https://doi.org/10.1126/science.aec8352).
- [24] J. Jumper, R. Evans, A. Pritzel u. a. „Highly accurate protein structure prediction with AlphaFold“. In: *Nature* 596 (2021), S. 583–589. DOI: [10.1038/s41586-021-03819-2](https://doi.org/10.1038/s41586-021-03819-2).
- [25] A. Davies, P. Veličković, L. Buesing u. a. „Advancing mathematics by guiding human intuition with AI“. In: *Nature* 600 (2021), S. 70–74. DOI: [10.1038/s41586-021-04086-x](https://doi.org/10.1038/s41586-021-04086-x).
- [26] B. Romera-Paredes, M. Barekatain, A. Novikov u. a. „Mathematical discoveries from program search with large language models“. In: *Nature* 625 (2024), S. 468–475. DOI: [10.1038/s41586-023-06924-6](https://doi.org/10.1038/s41586-023-06924-6).
- [27] T. Hubert, R. Mehta, L. Sartran u. a. „Olympiad-level formal mathematical reasoning with reinforcement learning“. In: *Nature* 651.8106 (2026). Online veröffentlicht am 12.11.2025, S. 607–613. DOI: [10.1038/s41586-025-09833-y](https://doi.org/10.1038/s41586-025-09833-y).
- [28] D. E. Knuth. *Claude’s Cycles*. Vom 28.02.2026, überarbeitet am 14.04.2026. 2026. URL: <https://cs.stanford.edu/~knuth/papers/claude-cycles.pdf> (besucht am 13.08.2026).
- [29] Anthropic. *Learning more about Claude’s mathematical capabilities*. Vom 10.08.2026. 2026. URL: <https://www.anthropic.com/research/riemann-zeta> (besucht am 13.08.2026).
- [30] OpenAI. *An OpenAI model has disproved a central conjecture in discrete geometry*. Vom 20.05.2026. 2026. URL: <https://openai.com/index/model-disproves-discrete-geometry-conjecture/> (besucht am 13.08.2026).
- [31] M. D. Schwartz. *Resummation of the C-Parameter Sudakov Shoulder Using Effective Field Theory*. arXiv:2601.02484, eingereicht am 05.01.2026. 2026. URL: <https://arxiv.org/abs/2601.02484> (besucht am 13.08.2026).
- [32] *Rewriting Bun in Rust*. Oven (Bun). 2026. URL: <https://bun.com/blog/bun-in-rust> (besucht am 13.08.2026).
- [33] *Rewrite Bun in Rust*. oven-sh/bun, Pull Request 30412. Gemergt am 14.05.2026; 1.009.257 hinzugefügte Zeilen. Metadaten per GitHub-API verifiziert. GitHub. 2026. URL: <https://github.com/oven-sh/bun/pull/30412> (besucht am 13.08.2026).

- [34] *PathString::slice dangling reference UB - add Miri to CI (oven-sh/bun, Issue 30719)*. Gemeldet am 14.05.2026, nach dem Merge des Rust-Ports. GitHub. 2026. URL: <https://github.com/oven-sh/bun/issues/30719> (besucht am 13.08.2026).
- [35] M. Miller. *Trends, Challenges, and Strategic Shifts in the Software Vulnerability Mitigation Landscape*. Vortrag, BlueHat IL 2019, Tel Aviv. Microsoft Security Response Center, BlueHat IL 2019. 7. Feb. 2019. URL: <https://www.youtube.com/watch?v=PjbGojJnBZQ> (besucht am 31.05.2026).
- [36] The Chromium Project. *Memory safety*. The Chromium Project. URL: <https://www.chromium.org/Home/chromium-security/memory-safety/> (besucht am 31.05.2026).
- [37] National Security Agency. *Software Memory Safety. Cybersecurity Information Sheet*. National Security Agency (NSA). 10. Nov. 2022. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF (besucht am 31.05.2026).
- [38] The Rust Project. *Rust Programming Language*. 2025. URL: <https://www.rust-lang.org/> (besucht am 16.02.2026).
- [39] S. Klabnik und C. Nichols. *The Rust Programming Language*. 2025. URL: <https://doc.rust-lang.org/book/> (besucht am 16.02.2026).
- [40] The Rust Project. *The Rust Reference*. 2025. URL: <https://doc.rust-lang.org/reference/> (besucht am 16.02.2026).
- [41] ISO/IEC JTC 1/SC 22/WG14. *Programming languages: C*. N2310. Frei verfügbarer Arbeitsentwurf zu ISO/IEC 9899:2018 (C17). ISO/IEC JTC 1/SC 22/WG14. 2018. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (besucht am 31.05.2026).
- [42] Zig Software Foundation. *Zig Language Reference*. Version 0.16.0. Zig Software Foundation. 2026. URL: <https://ziglang.org/documentation/master/> (besucht am 15.05.2026).
- [43] J. Sumner. *40 memory leaks and dozens of crashes fixed this way in bun v1.3.14*. X (vormals Twitter). 2026. URL: <https://x.com/jarredsumner/status/2055934488526619129> (besucht am 31.05.2026).
- [44] pnpm-Projekt. *pacquet*. Offizielle Rust-Portierung der Installations-Engine von pnpm; Repositorium archiviert am 21.07.2026, Weiterentwicklung im Hauptrepositorium von pnpm. GitHub. 2026. URL: <https://github.com/pnpm/pacquet> (besucht am 13.08.2026).

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus Veröffentlichungen oder aus anderweitigen fremden Quellen entnommen wurden, sind als solche kenntlich gemacht.

Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

.....
Ort, Datum

.....
Unterschrift